

Lab 3

Federation by SAML

Devanshu Trivedi

IAM Security Analyst

Environment: Ping Identity DevOps Docker Lab - PingDirectory · PingFederate · PingAccess · Cascade Internal Portal

August 2026

Table of Contents

1. Objective	3
2. Step 1: Creating the SAML Service Provider Connection.....	3
3. Step 2: Exporting PingFederate IdP Metadata.....	22
4. Step 3: Configuring the Cascade Portal for SAML.....	25
5. Summary	37

1. Objective

This lab connects the Cascade Internal Portal to PingFederate using SAML 2.0, completing the identity chain built across Labs 1 through 3. Where Lab 1 built the CyberCorp directory and Lab 2 built PingFederate's authentication infrastructure, Lab 3 builds the actual federation trust relationship: a SAML Service Provider (SP) Connection that tells PingFederate how to talk to Cascade, an exported IdP metadata document that gives Cascade what it needs to trust PingFederate, and the application-side configuration that makes Cascade actually redirect users into this flow and accept the result.

Every explanation in this document is written in detail on purpose - the goal is a reference you can come back to later and reconstruct not just what was clicked, but why each setting was chosen, so the reasoning is preserved alongside the steps.

2. Step 1: Creating the SAML Service Provider Connection

The first major configuration in this lab was creating a Service Provider Connection in PingFederate for the Cascade Internal Portal. A Service Provider Connection defines how PingFederate communicates with a specific SAML-enabled application and establishes that application's identity and protocol configuration. Conceptually, this configuration tells PingFederate that Cascade is an external application that trusts PingFederate to authenticate users and issue SAML assertions - instead of Cascade independently authenticating users against PingDirectory, Cascade relies on PingFederate to perform authentication and hand back an authenticated identity.

Element	Value
Connection Name	Cascade-Internal-Portal
Partner's Entity ID (final)	urn:cascade:internal-portal
Protocol	SAML 2.0, Browser SSO Profiles
SSO Profile	SP-Initiated SSO only
Authentication Policy Contract used	CyberCorp-User-Authentication (Lab 2)
Attribute Contract	SAML_SUBJECT, departmentNumber, givenName, mail, uid, username
ACS URL	https://192.168.131.160:3000/auth/saml/callback
Signing Certificate	CN=PingFederate Signing Certificate (...01:A0:28:54:13:3A)

2.1 Starting the Connection - Type & Options

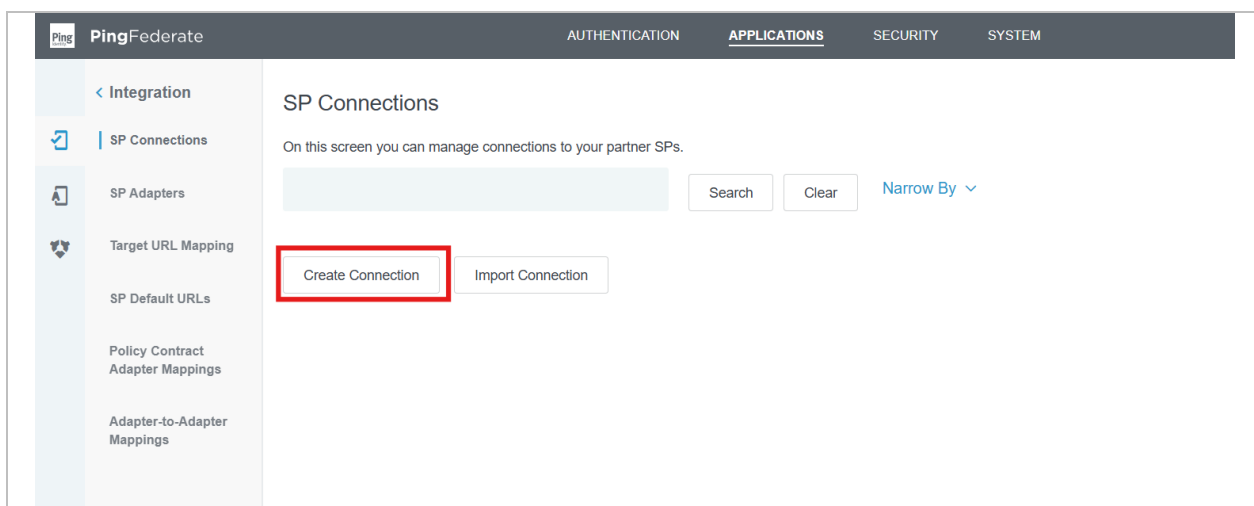


Figure 1. SP Connections list; click Create Connection.

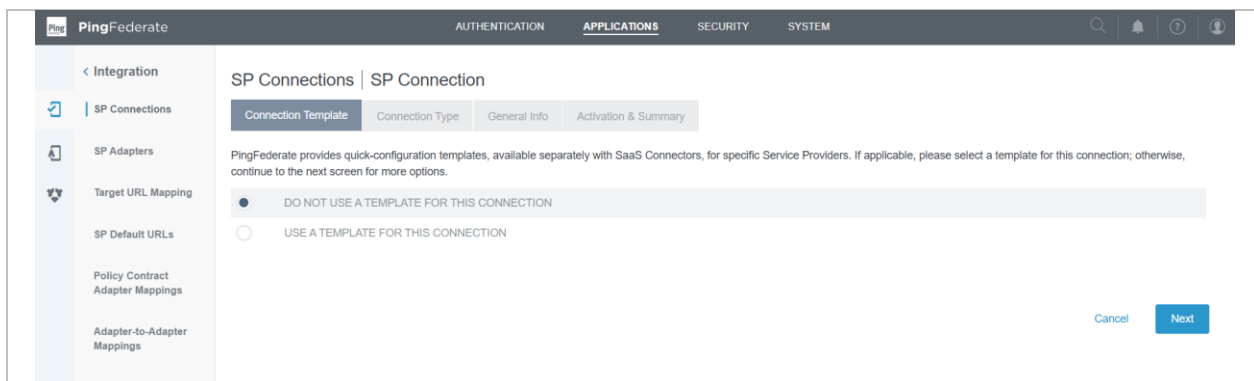


Figure 2. Connection Template: do not use a template for this connection.

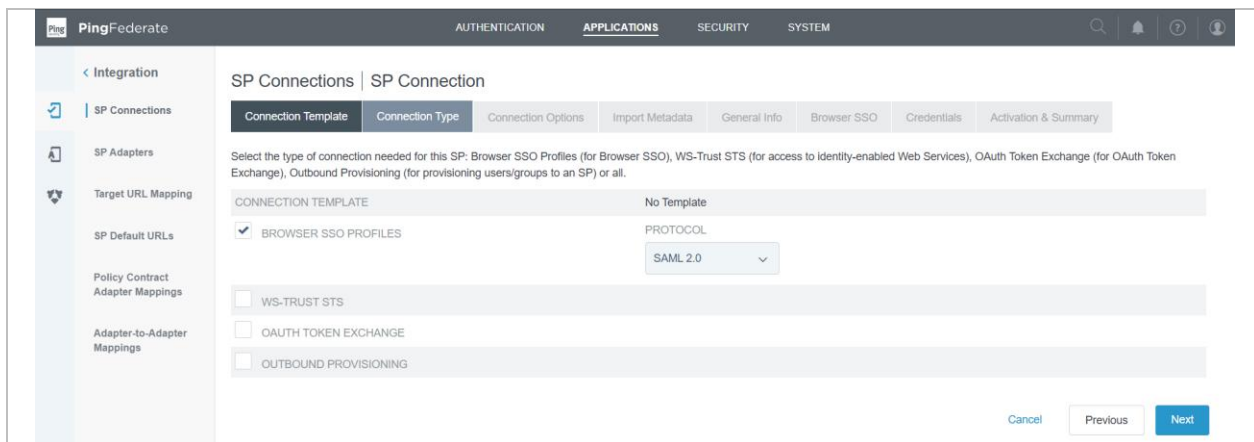
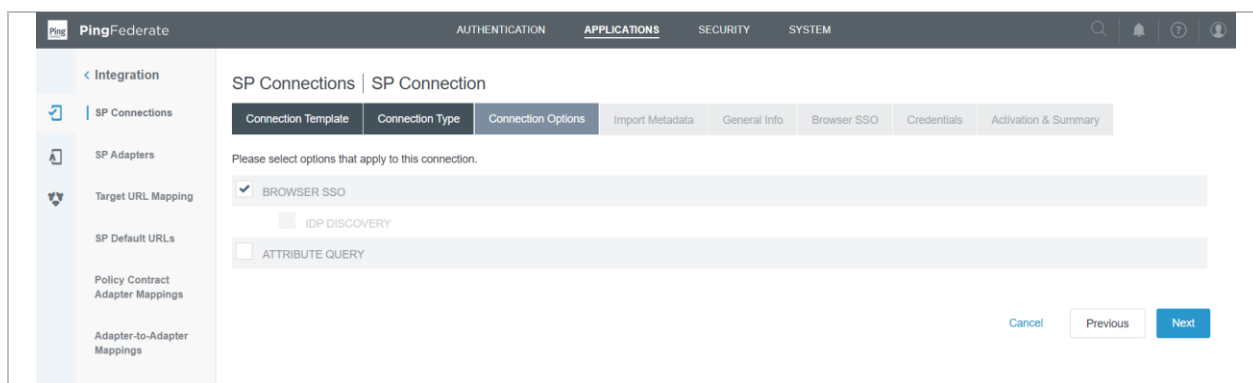


Figure 3. Connection Type: Browser SSO Profiles checked, protocol SAML 2.0.



The screenshot shows the PingFederate web interface. The top navigation bar includes 'AUTHENTICATION', 'APPLICATIONS', 'SECURITY', and 'SYSTEM'. The left sidebar shows 'Integration' with sub-items like 'SP Connections', 'SP Adapters', 'Target URL Mapping', 'SP Default URLs', 'Policy Contract Adapter Mappings', and 'Adapter-to-Adapter Mappings'. The main content area is titled 'SP Connections | SP Connection' and has tabs for 'Connection Template', 'Connection Type', 'Connection Options', 'Import Metadata', 'General Info', 'Browser SSO', 'Credentials', and 'Activation & Summary'. The 'Connection Options' tab is active, showing a section 'Please select options that apply to this connection.' with three options: 'BROWSER SSO' (checked), 'IDP DISCOVERY' (disabled), and 'ATTRIBUTE QUERY' (unchecked). At the bottom right are 'Cancel', 'Previous', and 'Next' buttons.

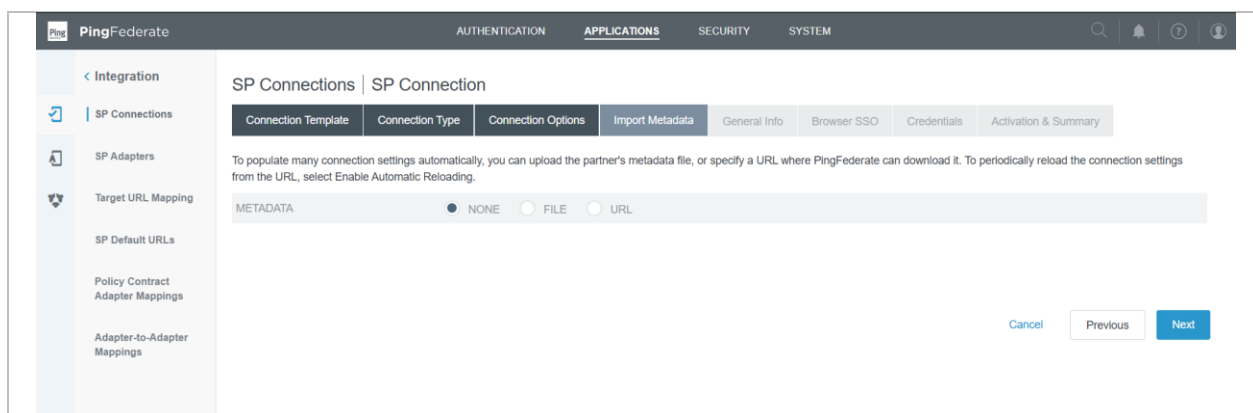
Figure 4. Connection Options: Browser SSO checked; IdP Discovery unavailable; Attribute Query left unchecked.

What we did and why

We started from SP Connections → Create Connection and chose not to use a quick-configuration template, since Cascade is a custom application rather than a pre-integrated SaaS connector. Templates exist for well-known SaaS products and pre-fill settings that don't apply to a homegrown Node.js app, so building the connection from scratch keeps every setting intentional and understood.

On Connection Type we selected Browser SSO Profiles with the SAML 2.0 protocol, and left WS-Trust STS, OAuth Token Exchange, and Outbound Provisioning unchecked. Browser SSO is the profile family covering standard web-based single sign-on - a user's browser is redirected between the SP and IdP to complete authentication - which is exactly the flow Cascade needs. On Connection Options, Browser SSO stayed checked (IdP Discovery isn't relevant since there's only one IdP in this environment) and Attribute Query was left unchecked, since Cascade doesn't need to separately query PingFederate for attributes outside of the SSO flow itself.

2.2 General Info - Naming the Partner



The screenshot shows the PingFederate web interface with the 'Import Metadata' tab selected. The main content area has a heading 'To populate many connection settings automatically, you can upload the partner's metadata file, or specify a URL where PingFederate can download it. To periodically reload the connection settings from the URL, select Enable Automatic Reloading.' Below this is a 'METADATA' section with three radio buttons: 'NONE' (selected), 'FILE', and 'URL'. At the bottom right are 'Cancel', 'Previous', and 'Next' buttons.

Figure 5. Import Metadata: Metadata set to None - everything is configured manually.

PingFederate AUTHENTICATION APPLICATIONS SECURITY SYSTEM

Integration

- SP Connections
- SP Adapters
- Target URL Mapping
- SP Default URLs
- Policy Contract Adapter Mappings
- Adapter-to-Adapter Mappings

SP Connections | SP Connection

Connection Template | Connection Type | Connection Options | Import Metadata | **General Info** | Browser SSO | Credentials | Activation & Summary

This information identifies your partner's unique connection identifier (Connection ID). Connection Name represents the plain-language identifier for this connection. Optionally, you can specify multiple virtual server IDs for your own server to use when communicating with this partner. If set, these virtual server IDs will be used in place of the unique protocol identifier configured for your server in Server Settings. The Base URL may be used to simplify configuration of partner endpoints.

PARTNER'S ENTITY ID (CONNECTION ID)

CONNECTION NAME

VIRTUAL SERVER IDS

BASE URL

COMPANY

CONTACT NAME

CONTACT NUMBER

CONTACT EMAIL

APPLICATION NAME

APPLICATION ICON URL

TRANSACTION LOGGING

Figure 6. General Info: Partner's Entity ID CascadeInternalPortal, Connection Name Cascade-Internal-Portal, Company CyberCorp, Application Name Cascade Internal Portal.

What we did and why

Metadata import was left as None. PingFederate can auto-populate much of a connection's configuration from a partner's SAML metadata file or URL, but because Cascade is a custom application with no metadata endpoint of its own at this stage, every setting was entered manually instead.

On General Info we entered a Partner's Entity ID (Connection ID) of CascadeInternalPortal, a Connection Name of Cascade-Internal-Portal, Company CyberCorp, and Application Name Cascade Internal Portal.

The Partner's Entity ID is the single most important value on this screen: it's the unique identifier PingFederate uses to look up this connection whenever a SAML request or response references it. Whatever value goes here must exactly match the issuer value the Cascade application sends in its AuthnRequest - a mismatch here is exactly what caused Error 1 later in this lab (see 4.5). At this point in the lab, that mismatch hadn't been discovered yet.

2.3 Browser SSO - SAML Profiles

SP Connections | SP Connection

Connection Template | Connection Type | Connection Options | Import Metadata | General Info | **Browser SSO** | Credentials | Activation & Summary

This task provides connection-endpoint and other configuration information enabling secure browser-based SSO, to resources at your partner's site. Click the button below to create or revise this configuration.

BROWSER SSO CONFIGURATION

Configure Browser SSO

Cancel Save Draft Previous Next

Figure 7. Browser SSO tab, before configuration - Configure Browser SSO.

SP Connections | SP Connection | **Browser SSO**

SAML Profiles | Assertion Lifetime | Assertion Creation | Protocol Settings | Summary

A SAML Profile defines what kind of messages may be exchanged between an Identity Provider and a Service Provider, and how the messages are transported (bindings). As an IdP, you configure this information for your SP connection.

Single Sign-On (SSO) Profiles | Single Logout (SLO) Profiles

☐ IDP-INITIATED SSO ☐ IDP-INITIATED SLO

☒ SP-INITIATED SSO ☐ SP-INITIATED SLO

Cancel Save Draft Next

Figure 8. SAML Profiles: SP-Initiated SSO checked; IdP-Initiated SSO, IdP-Initiated SLO, and SP-Initiated SLO left unchecked.

What we did and why

Within the Service Provider Connection, the Browser SSO profile was configured to support an SP-initiated SSO flow. SP-initiated SSO means the authentication transaction begins when a user attempts to access the Cascade application, and the application redirects the user to PingFederate for authentication. In this architecture, Cascade acts as the SAML Service Provider and PingFederate acts as the Identity Provider: Cascade initiates a SAML AuthnRequest and directs the user's browser to PingFederate, PingFederate processes the request and invokes the authentication policy configured in Lab 2, and after successful authentication PingFederate produces a SAML response and returns it to Cascade.

We enabled only SP-Initiated SSO because it represents the primary authentication flow for our internal portal. IdP-initiated SSO would let a user start at PingFederate and be pushed into Cascade without Cascade asking first, and Single Logout (SLO) would propagate a logout across every federated application - both are reasonable additions for a later lab, but out of scope for establishing the basic SSO workflow here.

2.4 Browser SSO - Assertion Lifetime

SP Connections | SP Connection | Browser SSO

SAML Profiles | Assertion Lifetime | Assertion Creation | Protocol Settings | Summary

When an assertion is issued to the SP, there is a timeframe of validity before and after issuance. Please specify these parameters below.

MINUTES BEFORE	5
MINUTES AFTER	5

Figure 9. Assertion Lifetime: 5 minutes before and 5 minutes after issuance.

What we did and why

The SAML assertion lifetime was left at PingFederate's default values: 5 minutes before and 5 minutes after the assertion's issuance time. An assertion lifetime defines the window during which a SAML assertion is considered valid - it protects against an assertion being accepted indefinitely and reduces the impact of an old assertion being replayed by an attacker who somehow captured it.

The 5-minutes-before / 5-minutes-after configuration allows a small amount of clock tolerance between PingFederate and Cascade while keeping the validity window relatively short. This is appropriate for a lab because it demonstrates the concept of assertion validity without introducing unnecessary complexity. In a production environment this window would typically be tuned based on the organization's security requirements, how tightly server clocks are synchronized (for example via NTP), and the application's own risk tolerance.

2.5 Assertion Creation - Identity Mapping & Attribute Contract

SP Connections | SP Connection | Browser SSO

SAML Profiles | Assertion Lifetime | Assertion Creation | Protocol Settings | Summary

This task provides the configuration for creating SAML assertions to enable SSO access to resources at your SP partner's site.

Assertion Configuration

IDENTITY MAPPING	Standard
ATTRIBUTE CONTRACT	SAML_SUBJECT
ADAPTER INSTANCES	0
AUTHENTICATION POLICY MAPPINGS	0

Configure Assertion Creation

Figure 10. Assertion Creation tab, before configuration - Configure Assertion Creation.

SP Connections | SP Connection | Browser SSO | Assertion Creation

Identity Mapping | Attribute Contract | Authentication Source Mapping | Summary

Identity mapping is the process in which users authenticated by the IdP are associated with user accounts local to the SP. Select the type of name identifier that you will send to the SP. Your selection may affect the way that the SP will look up and associate the user to a specific local account.

☒ **STANDARD:** Send the SP a known attribute value as the name identifier. The SP will often use account mapping to identify the user locally.

☐ **PSEUDONYM:** Send the SP a unique, opaque name identifier that preserves user privacy. The identifier cannot be traced back to the user's identity at this IdP and may be used by the SP to make a persistent association between the user and a specific local account. The SP will often use account linking to identify the user locally.

☐ INCLUDE ATTRIBUTES IN ADDITION TO THE PSEUDONYM.

☐ **TRANSIENT:** Send the SP an opaque, temporary value as the name identifier.

☐ INCLUDE ATTRIBUTES IN ADDITION TO THE TRANSIENT IDENTIFIER.

[Cancel](#) [Save Draft](#) [Next](#)

Figure 11. Identity Mapping: Standard selected.

SP Connections | SP Connection | Browser SSO | Assertion Creation

Identity Mapping | Attribute Contract | Authentication Source Mapping | Summary

An Attribute Contract is a set of user attributes that this server will send in the assertion.

Attribute Contract **Subject Name Format**

SAML_SUBJECT

Extend the Contract	Attribute Name Format	Action
departmentNumber	urn:oasis:names:tc:SAML:2.0:attrname-format:basic	Edit Delete
givenName	urn:oasis:names:tc:SAML:2.0:attrname-format:basic	Edit Delete
mail	urn:oasis:names:tc:SAML:2.0:attrname-format:basic	Edit Delete
uid	urn:oasis:names:tc:SAML:2.0:attrname-format:basic	Edit Delete
username	urn:oasis:names:tc:SAML:2.0:attrname-format:basic	Edit Delete
	urn:oasis:names:tc:SAML:2.0:attrname-format:basic	Add

[Cancel](#) [Save Draft](#) [Previous](#) [Next](#)

Figure 12. Attribute Contract: SAML_SUBJECT plus departmentNumber, givenName, mail, uid, and username, using the SAML 2.0 basic attribute name format.

What we did and why

The Assertion Creation configuration determines what PingFederate places inside the SAML assertion after a user has successfully authenticated. The assertion is the security statement PingFederate makes to Cascade: in effect, PingFederate is telling Cascade that a particular user successfully authenticated, and handing over the identity information Cascade is allowed to consume.

Identity Mapping was configured as Standard, meaning PingFederate sends Cascade a known attribute value as the name identifier, rather than Pseudonym (an opaque, privacy-preserving identifier the SP can't trace back to the user) or Transient (a temporary, one-time-use identifier). Standard was the right choice because Cascade needs a consistent, recognizable way to associate the SAML identity with the correct application account - the same user should map to the same Cascade identity every time they sign in, not a different opaque value on every login.

The Attribute Contract - the set of user attributes PingFederate promises to send in every assertion - was extended beyond the default SAML_SUBJECT to include departmentNumber, givenName, mail, uid, and username, all using the SAML 2.0 basic attribute name format. These five attributes were chosen because they were already flowing through the authentication and attribute-retrieval configuration completed in Lab 2 - nothing new needed to be looked up. Defining an explicit contract, rather than forwarding arbitrary LDAP attributes, means the Service Provider receives only the identity information the application actually needs: a predictable, minimal, security-conscious data shape.

The Subject Name Format was left at the default urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified, meaning the assertion doesn't impose a more specific SAML NameID format such as persistent or transient at this stage. The actual subject value is supplied later, from the Authentication Policy Contract (see 2.6) - creating a clear chain from the user's LDAP identity, through PingFederate's authentication process, to the identity ultimately presented to Cascade.

2.6 Assertion Creation - Authentication Source Mapping

SP Connections | SP Connection | Browser SSO | Assertion Creation

Identity Mapping | Attribute Contract | **Authentication Source Mapping** | Summary

PingFederate uses IdP adapters, partner IdPs or Authentication Policies to authenticate users to your SP. Users may be authenticated by one of several different adapters or Policy Contracts, so map an adapter instance for each IDM system or an authentication policy contract for each policy.

Adapter Instance Name	Virtual Server IDs	Action

Authentication Policy Contract Name	Virtual Server IDs	Action

Map New Adapter Instance | **Map New Authentication Policy**

Figure 13. Authentication Source Mapping, empty - Map New Authentication Policy.

[SP Connections](#) | [SP Connection](#) | [Browser SSO](#) | [Assertion Creation](#) | [Authentication Policy Mapping](#)

[Authentication Policy Contract](#) | [Mapping Method](#) | [Attribute Contract Fulfillment](#) | [Issuance Criteria](#) | [Summary](#)

Select an authentication policy contract that may be used to authenticate users for this partner. Attributes returned by the authentication policy contract you choose may be used to fulfill the Attribute Contract with your partner.

AUTHENTICATION POLICY CONTRACT

CyberCorp-User-Authentication

Contract Attributes

cn

departmentNumber

givenName

mail

subject

title

uid

username

Manage Policy Contracts

[Cancel](#) [Save Draft](#) [Next](#)

Figure 14. Authentication Policy Contract selected: CyberCorp-User-Authentication, showing its full attribute list.

[SP Connections](#) | [SP Connection](#) | [Browser SSO](#) | [Assertion Creation](#) | [Authentication Policy Mapping](#)

[Authentication Policy Contract](#) | [Mapping Method](#) | [Attribute Contract Fulfillment](#) | [Issuance Criteria](#) | [Summary](#)

You can choose to fulfill the Attribute Contract with your partner using either the values provided by the "CyberCorp-User-Authentication" authentication policy contract, or you can use these values plus additional attributes retrieved from local data stores.

Authentication Policy Contract

cn

departmentNumber

givenName

mail

subject

title

uid

username

☐ RETRIEVE ADDITIONAL ATTRIBUTES FROM MULTIPLE DATA STORES USING ONE MAPPING

☐ RETRIEVE ADDITIONAL ATTRIBUTES FROM A DATA STORE – INCLUDES OPTIONS TO USE ALTERNATE DATA STORES AND/OR A FAILSAFE MAPPING

☒ USE ONLY THE AUTHENTICATION POLICY CONTRACT VALUES IN THE SAML ASSERTION

Figure 15. Mapping Method: "Use only the Authentication Policy Contract values in the SAML assertion."

SP Connections | SP Connection | Browser SSO | Assertion Creation | Authentication Policy Mapping

Authentication Policy Contract | Mapping Method | Attribute Contract Fulfillment | Issuance Criteria | Summary

Fulfill your Attribute Contract with values from the authentication policy contract or with dynamic text values.

Attribute Contract	Source	Value ?	Actions
SAML_SUBJECT	Authentication Policy Contract	subject	None available
departmentNumber	Authentication Policy Contract	departmentNumber	None available
givenName	Authentication Policy Contract	givenName	None available
mail	Authentication Policy Contract	mail	None available
uid	Authentication Policy Contract	uid	None available
username	Authentication Policy Contract	username	None available

Figure 16. Attribute Contract Fulfillment: every SAML attribute mapped from the Authentication Policy Contract.

SP Connections | SP Connection | Browser SSO | Assertion Creation | Authentication Policy Mapping

Authentication Policy Contract | Mapping Method | Attribute Contract Fulfillment | Issuance Criteria | Summary

PingFederate can evaluate various criteria to determine whether users are authorized to access SP resources. Use this optional screen to configure the criteria for use with this conditional authorization.

Source	Attribute Name	Condition	Value	Error Result	Action
- SELECT -	- SELECT -	- SELECT -			Add

Cancel Save Draft Previous Next

Figure 17. Issuance Criteria left unconfigured.

SP Connections | SP Connection | Browser SSO | Assertion Creation | Authentication Policy Mapping

Authentication Policy Contract | Mapping Method | Attribute Contract Fulfillment | Issuance Criteria | Summary

Click a heading link to edit a configuration setting.

Authentication Policy Contract

Selected contract	CyberCorp-User-Authentication
-------------------	-------------------------------

Mapping Method

Authentication Policy Contract	CyberCorp-User-Authentication
Mapping Method	Use only the Authentication Policy Contract values in the mapping

Attribute Contract Fulfillment

SAML_SUBJECT	subject (Authentication Policy Contract)
departmentNumber	departmentNumber (Authentication Policy Contract)
givenName	givenName (Authentication Policy Contract)
mail	mail (Authentication Policy Contract)
uid	uid (Authentication Policy Contract)
username	username (Authentication Policy Contract)

Issuance Criteria

Criterion	(None)
-----------	--------

Cancel Save Draft Previous Done

Figure 18. Authentication Policy Mapping summary before returning to Assertion Creation.

SP Connections | SP Connection | Browser SSO | Assertion Creation

Identity Mapping | Attribute Contract | Authentication Source Mapping | Summary

PingFederate uses IdP adapters, partner IdPs or Authentication Policies to authenticate users to your SP. Users may be authenticated by one of several different adapters or Policy Contracts, so map an adapter instance for each IDM system or an authentication policy contract for each policy.

Adapter Instance Name	Virtual Server IDs	Action
Authentication Policy Contract Name	Virtual Server IDs	Action
CyberCorp-User-Authentication		Delete

Map New Adapter Instance Map New Authentication Policy

Cancel Save Draft Previous Next

Figure 19. Authentication Source Mapping, now showing CyberCorp-User-Authentication mapped in.

PingFederate

AUTHENTICATION
APPLICATIONS
SECURITY
SYSTEM

Integration
SP Connections
SP Adapters
Target URL Mapping
SP Default URLs
Policy Contract Adapter Mappings
Adapter-to-Adapter Mappings

SP Connections | SP Connection | Browser SSO | Assertion Creation

Identity Mapping
Attribute Contract
Authentication Source Mapping
Summary

Summary information for your Assertion Creation configuration. Click a heading link to edit a configuration setting.

Assertion Creation

Identity Mapping

Enable Standard Identifier	true
----------------------------	------

Attribute Contract

Attribute	SAML_SUBJECT
Subject Name Format	urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified
Attribute	departmentNumber
Attribute Name Format	urn:oasis:names:tc:SAML:2.0:attrname-format:basic
Attribute	givenName
Attribute Name Format	urn:oasis:names:tc:SAML:2.0:attrname-format:basic
Attribute	mail
Attribute Name Format	urn:oasis:names:tc:SAML:2.0:attrname-format:basic
Attribute	uid
Attribute Name Format	urn:oasis:names:tc:SAML:2.0:attrname-format:basic
Attribute	username
Attribute Name Format	urn:oasis:names:tc:SAML:2.0:attrname-format:basic

Authentication Source Mapping

Authentication policy contract name	CyberCorp-User-Authentication
-------------------------------------	-------------------------------

Authentication Policy Contract

Selected contract	CyberCorp-User-Authentication
-------------------	-------------------------------

Mapping Method

Authentication Policy Contract	CyberCorp-User-Authentication
Mapping Method	Use only the Authentication Policy Contract values in the mapping

Attribute Contract Fulfilment

SAML_SUBJECT	subject (Authentication Policy Contract)
departmentNumber	departmentNumber (Authentication Policy Contract)
givenName	givenName (Authentication Policy Contract)
mail	mail (Authentication Policy Contract)
uid	uid (Authentication Policy Contract)
username	username (Authentication Policy Contract)

Issuance Criteria

Criterion	(None)
-----------	--------

Cancel
Save Draft
Previous
Done

Figure 20. Full Assertion Creation summary - Identity Mapping, Attribute Contract, Authentication Source Mapping, and Attribute Contract Fulfilment together.

What we did and why

The next configuration connected the new SAML Service Provider Connection to the Authentication Policy Contract created in Lab 2. Rather than building a new authentication mechanism specifically for Cascade, the connection was configured to reuse the existing CyberCorp-User-Authentication Authentication Policy Contract via Map New Authentication Policy.

This is one of the most important architectural features of PingFederate demonstrated in this lab: authentication logic can be built once and reused across every application that federates through PingFederate. The Cascade Portal doesn't need to know how PingDirectory works, how the LDAP Password Credential Validator works, or how the HTML Form Adapter collects credentials - all of that stays inside the Lab 2 authentication policy. The SAML connection simply consumes the authenticated result that policy produces.

The selected CyberCorp-User-Authentication contract carries `cn`, `departmentNumber`, `givenName`, `mail`, `subject`, `title`, `uid`, and `username`. For the Mapping Method, we selected "Use only the Authentication Policy Contract values in the SAML assertion" rather than the two alternatives that would retrieve additional attributes from a data store during the SAML transaction itself. This means PingFederate reuses the attributes already obtained during authentication instead of performing a second LDAP lookup while building the assertion - keeping the SSO transaction fast and avoiding duplicate directory queries. It also enforces a clean separation of responsibilities: PingDirectory is the authoritative identity source, the Lab 2 authentication policy performs authentication and attribute retrieval, and the SAML assertion-creation process here simply consumes the resulting contract values.

In Attribute Contract Fulfillment, each SAML attribute was mapped to its Authentication Policy Contract counterpart:

- `SAML_SUBJECT` $\leftarrow$ `subject`
- `departmentNumber` $\leftarrow$ `departmentNumber`
- `givenName` $\leftarrow$ `givenName`
- `mail` $\leftarrow$ `mail`
- `uid` $\leftarrow$ `uid`
- `username` $\leftarrow$ `username`

- all sourced from Authentication Policy Contract. This is the configuration that ultimately transforms internal authentication information into application-facing SAML data: once a user authenticates and the policy contract holds their username, email, first name, and department, those exact values flow into the SAML assertion and are handed to Cascade.

Issuance Criteria were intentionally left unconfigured. Issuance criteria let PingFederate apply additional conditions - for example, requiring a user to belong to a particular department or group before an assertion is issued - before granting SSO access. That kind of attribute-based authorization wasn't the goal of this initial SSO implementation and is left for a later, more advanced lab in the sequence.

2.7 Protocol Settings - ACS URL, Bindings, Signature & Encryption

SP Connections | SP Connection | Browser SSO

SAML Profiles | Assertion Lifetime | Assertion Creation | Protocol Settings | Summary

This task provides the configuration for creating SAML assertions to enable SSO access to resources at your SP partner's site.

Assertion Configuration

IDENTITY MAPPING	Standard
ATTRIBUTE CONTRACT	SAML_SUBJECT, departmentNumber, givenName, mail, uid, username
ADAPTER INSTANCES	0
AUTHENTICATION POLICY MAPPINGS	1

Configure Assertion Creation

Cancel Save Draft Previous Next

Figure 21. Assertion Creation tab, post-configuration.

SP Connections | SP Connection | Browser SSO

SAML Profiles | Assertion Lifetime | Assertion Creation | Protocol Settings | Summary

This task provides the configuration for specific endpoints and security considerations applicable to selected profiles. Click the button below to create or revise this configuration.

Protocol Settings

OUTBOUND SSO BINDINGS

INBOUND BINDINGS	POST, Redirect, Artifact, SOAP
SIGNATURE POLICY	SAML-standard
ENCRYPTION POLICY	No Encryption

Configure Protocol Settings

Cancel Save Draft Previous Next

Figure 22. Protocol Settings tab, before configuration - Configure Protocol Settings.

SP Connections | SP Connection | Browser SSO | Protocol Settings

Assertion Consumer Service URL | Allowable SAML Bindings | Artifact Resolver Locations | Signature Policy | Encryption Policy | Summary

As the IdP, you send SAML assertions to the SP's Assertion Consumer Service. The SP may request that the SAML assertion be sent to one of several URLs, via different bindings. Please provide the possible assertion consumer URLs below and select one to be the default.

Default	Index	Binding	Endpoint URL	Action
default	0	POST	https://192.168.131.160:3000/auth/saml/callback	Edit Delete
☐		- SELECT -		Add

Cancel Save Draft Next

Figure 23. Assertion Consumer Service URL: https://192.168.131.160:3000/auth/saml/callback, POST, set as default.

SP Connections | SP Connection | Browser SSO | Protocol Settings

Assertion Consumer Service URL | Allowable SAML Bindings | Signature Policy | Encryption Policy | Summary

When the SP sends messages, what SAML bindings do you want to allow?

☐ ARTIFACT

☒ POST

☒ REDIRECT

☐ SOAP

[Cancel](#) [Save Draft](#) [Previous](#) [Next](#)

Figure 24. Allowable SAML Bindings: POST and Redirect checked; Artifact and SOAP left unchecked.

SP Connections | SP Connection | Browser SSO | Protocol Settings

Assertion Consumer Service URL | Allowable SAML Bindings | Signature Policy | Encryption Policy | Summary

Additional guarantees of authenticity may be agreed upon between you and your partner. For SP-initiated SSO, you can choose to require signed authentication requests sent via the POST or redirect bindings. You can also choose to sign assertions sent to this partner, regardless of the binding used.

☐ REQUIRE AUTHN REQUESTS TO BE SIGNED WHEN RECEIVED VIA THE POST OR REDIRECT BINDINGS

☒ ALWAYS SIGN ASSERTION

☒ SIGN RESPONSE AS REQUIRED

[Cancel](#) [Save Draft](#) [Previous](#) [Next](#)

Figure 25. Signature Policy: Always Sign Assertion and Sign Response As Required checked; signed AuthnRequests not required.

SP Connections | SP Connection | Browser SSO | Protocol Settings

Assertion Consumer Service URL | Allowable SAML Bindings | Signature Policy | Encryption Policy | Summary

Encryption may be applied to SAML messages for an added layer of protection in transport. If enabled, SAML Response messages may always be signed, regardless of the signature policy.

☒ NONE

☐ THE ENTIRE ASSERTION

☐ ONE OR MORE ATTRIBUTES

☐ SAML_SUBJECT

☐ DEPARTMENTNUMBER

☐ GIVENNAME

☐ MAIL

☐ UID

☐ USERNAME

[Cancel](#) [Save Draft](#) [Previous](#) [Next](#)

Figure 26. Encryption Policy: None.

SP Connections | SP Connection | Browser SSO | Protocol Settings

Assertion Consumer Service URL | Allowable SAML Bindings | Signature Policy | Encryption Policy | Summary

Summary information for your Protocol Settings configuration. Click a heading link to edit a configuration setting.

Protocol Settings

Assertion Consumer Service URL

Endpoint	URL: https://192.168.131.160:3000/auth/saml/callback (POST)
----------	---

Allowable SAML Bindings

Artifact	false
POST	true
Redirect	true
SOAP	false

Signature Policy

Require digitally signed AuthN requests	false
Always Sign Assertion	true
Sign Response As Required	true

Encryption Policy

Status	Inactive
--------	----------

Figure 27. Protocol Settings summary.

SP Connections | SP Connection | Browser SSO

SAML Profiles | Assertion Lifetime | Assertion Creation | Protocol Settings | Summary

This task provides the configuration for specific endpoints and security considerations applicable to selected profiles. Click the button below to create or revise this configuration.

Protocol Settings

OUTBOUND SSO BINDINGS	POST
INBOUND BINDINGS	POST, Redirect
SIGNATURE POLICY	SAML-standard, SAML Assertion Signed
ENCRYPTION POLICY	No Encryption

Configure Protocol Settings

Cancel Save Draft Previous Next

Figure 28. Protocol Settings tab, post-configuration.

PingFederate

Authentication

Applications

Security

System

Search

Alerts

Help

Logout

Integration

SP Connections

SP Adapters

Target URL Mapping

SP Default URLs

Policy Contract Adapter Mappings

Adapter-to-Adapter Mappings

SP Connections | SP Connection | Browser SSO

SAML Profiles

Assertion Lifetime

Assertion Creation

Protocol Settings

Summary

Summary information for your Browser SSO configuration. Click a heading link to edit a configuration setting.

Browser SSO

SAML Profiles

IdP-Initiated SSOfalse

IdP-Initiated SLOfalse

SP-Initiated SSOtrue

SP-Initiated SLOfalse

Assertion Lifetime

Valid Minutes Before5

Valid Minutes After5

Assertion Creation

Identity Mapping

Enable Standard Identifiertrue

Attribute Contract

AttributeSAML_SUBJECT

Subject Name Formaturn:oasis:names:tc:SAML:1.1:nameid-format:unspecified

AttributedepartmentNumber

Attribute Name Formaturn:oasis:names:tc:SAML:2.0:attribute-format:basic

AttributegivenName

Attribute Name Formaturn:oasis:names:tc:SAML:2.0:attribute-format:basic

Attributemail

Attribute Name Formaturn:oasis:names:tc:SAML:2.0:attribute-format:basic

Attributeuid

Attribute Name Formaturn:oasis:names:tc:SAML:2.0:attribute-format:basic

Attributeusername

Attribute Name Formaturn:oasis:names:tc:SAML:2.0:attribute-format:basic

Authentication Source Mapping

Authentication policy contract nameCyberCorp-User-Authentication

Authentication Policy Contract

Selected contractCyberCorp-User-Authentication

Mapping Method

Authentication Policy ContractCyberCorp-User-Authentication

Mapping MethodUse only the Authentication Policy Contract values in the mapping

Attribute Contract Fulfillment

SAML_SUBJECTsubject (Authentication Policy Contract)

departmentNumberdepartmentNumber (Authentication Policy Contract)

givenNamegivenName (Authentication Policy Contract)

mailmail (Authentication Policy Contract)

uiduid (Authentication Policy Contract)

usernameusername (Authentication Policy Contract)

Issuance Criteria

Criterion(None)

Protocol Settings

Assertion Consumer Service URL

EndpointURL: https://192.168.131.160:3000/auth/saml/callback (POST)

Allowable SAML Bindings

Artifactfalse

POSTtrue

Redirecttrue

SOAPfalse

Signature Policy

Require digitally signed AuthN requestsfalse

Always Sign Assertiontrue

Sign Response As Requiredtrue

Encryption Policy

StatusInactive

Cancel

Save Draft

Previous

Done

Figure 29. Full Browser SSO summary - SAML Profiles, Assertion Lifetime, Assertion Creation, and Protocol Settings together, before clicking Done.

What we did and why

A critical part of the SAML configuration was the Assertion Consumer Service (ACS) URL - the location where the Service Provider expects to receive the SAML response from the Identity Provider. For the Cascade Portal, the configured ACS endpoint was `https://192.168.131.160:3000/auth/saml/callback`. When authentication succeeds, PingFederate sends the SAML response to this URL through the user's browser; Cascade then processes the response, validates the assertion, extracts the identity information, and establishes the application's authenticated session. This URL is one of the most important pieces of the whole integration, because PingFederate must know exactly where to deliver the authentication result - and it must correspond exactly to the callback route the Cascade application actually implements.

The allowable SAML bindings were set to HTTP-Redirect and HTTP-POST, with Artifact and SOAP left disabled. SAML bindings determine how SAML messages are transported between the SP and IdP. HTTP-Redirect is appropriate for carrying the (comparatively small) authentication request from Cascade to PingFederate through the browser's URL; HTTP-POST is used to deliver the larger SAML response - which contains the full signed assertion - to Cascade's ACS endpoint via an auto-submitting HTML form. The resulting flow is conceptually: Cascade sends an AuthnRequest via HTTP redirect, PingFederate authenticates the user, and PingFederate returns the SAML response via HTTP POST to the Cascade callback URL. Artifact (which requires a back-channel SOAP call to retrieve the actual assertion) and SOAP were unnecessary for this browser-based flow.

The Signature Policy was configured so PingFederate always signs the assertion (Always Sign Assertion and Sign Response As Required were both checked), while requiring digitally signed AuthnRequests was left unchecked. This means the primary integrity and authenticity protection in this lab is applied to the assertion PingFederate produces, not to the request Cascade sends. A digitally signed assertion lets Cascade verify that the assertion genuinely originated from PingFederate and hasn't been tampered with in transit. Not requiring a signed AuthnRequest simplifies the lab, because Cascade isn't required to hold and use its own signing key just to ask PingFederate to authenticate a user - a production deployment might choose to require this depending on its trust model.

The Encryption Policy was set to None, meaning the SAML assertion body itself is not encrypted. This doesn't leave the assertion unprotected - signing still provides integrity and authenticity - but encryption, which provides confidentiality, was intentionally left out for this initial lab so the focus could stay on establishing a functional, verifiable SAML trust relationship. Assertion or per-attribute encryption is a natural next step when hardening this integration for production, particularly if the assertion carries sensitive identity attributes that shouldn't be readable beyond the intended Service Provider.

2.8 Credentials - Digital Signature

The screenshot shows the PingFederate interface with the 'APPLICATIONS' tab selected. The left sidebar shows the 'Integration' section with 'SP Connections' highlighted. The main content area is titled 'SP Connections | SP Connection' and has tabs for 'Connection Template', 'Connection Type', 'Connection Options', 'Import Metadata', 'General Info', 'Browser SSO', 'Credentials', and 'Activation & Summary'. The 'Browser SSO' tab is active, displaying a 'BROWSER SSO CONFIGURATION' section with a 'Configure Browser SSO' button. At the bottom right are 'Cancel', 'Save Draft', 'Previous', and 'Next' buttons.

Figure 30. Back at the main SP Connection wizard, Browser SSO tab.

The screenshot shows the 'Credentials' tab in the 'SP Connection' wizard. The 'Credentials' tab is active, showing a 'Credential Requirement' section with 'DIGITAL SIGNATURE' listed as 'Not Configured'. A red box highlights the 'Configure Credentials' button. The bottom right contains 'Cancel', 'Save Draft', 'Previous', and 'Next' buttons.

Figure 31. Credentials tab: Digital Signature Not Configured - Configure Credentials.

The screenshot shows the 'Digital Signature Settings' page. It has two tabs: 'Digital Signature Settings' (active) and 'Summary'. The page contains the following fields:

- SIGNING CERTIFICATE:** A dropdown menu showing 'CN=PingFederate Signing Certificate... (01:A0:28:54:13:3A | Exp: Aug 22, 2027)'.
- SECONDARY SIGNING CERTIFICATE:** A dropdown menu showing '- SELECT -'.
- INCLUDE THE CERTIFICATE IN THE SIGNATURE <KEYINFO> ELEMENT:** A checked checkbox.
- INCLUDE THE RAW KEY IN THE SIGNATURE <KEYVALUE> ELEMENT:** An unchecked checkbox.
- SIGNING ALGORITHM:** A dropdown menu showing 'RSA SHA256'.

 At the bottom left is a 'Manage Certificates' button.

Figure 32. Digital Signature Settings: PingFederate signing certificate selected, certificate included in KeyInfo, RSA SHA256.

SP Connections | SP Connection | Credentials

Digital Signature Settings | Summary

Summary information for your Credentials configuration. Click a heading link to edit a configuration setting.

Credentials	
Digital Signature Settings	
Selected Certificate	CN=PingFederate Signing Certificate, O=CyberCorp, C=US (01:A0:28:54:13:3A Exp: Aug 22, 2027)
Include Certificate in KeyInfo	true
Include Raw Key in KeyValue	false
Selected Signing Algorithm	RSA SHA256

Figure 33. Credentials summary.

PingFederate AUTHENTICATION APPLICATIONS SECURITY SYSTEM

< Integration

SP Connections | SP Connection

Connection Template | Connection Type | Connection Options | Import Metadata | General Info | Browser SSO | Credentials | Activation & Summary

For each credential shown here, configure the necessary settings.

Credential Requirement

DIGITAL SIGNATURE CN=PingFederate Signing Certificate, O=CyberCorp, C=US (01:A0:28:54:13:3A | Exp: Aug 22, 2027)

Configure Credentials

Cancel Save Draft Previous Next

Figure 34. Main wizard Credentials tab, post-configuration, before Activation & Summary.

What we did and why

Because the assertion was configured to always be signed, PingFederate required a digital signing certificate before the connection could be completed. We selected the existing PingFederate signing certificate (CN=PingFederate Signing Certificate, O=CyberCorp, C=US), identified by the fingerprint ending in 01:A0:28:54:13:3A and expiring August 22, 2027, using the RSA SHA256 signing algorithm.

unchecked. Including the certificate in KeyInfo lets the receiving Service Provider extract the public key it needs to validate the signature directly from the assertion itself, which simplifies trust setup on the SP side. The private key associated with this certificate never leaves PingFederate; it's used internally to produce the cryptographic signature, while only the public certificate is shared.

With Digital Signature Settings configured and confirmed on the Credentials summary, the connection moved to Activation & Summary, where it was reviewed and enabled - completing the SP Connection and making CyberCorp-User-Authentication, via Cascade-Portal-Login, available to authenticate users for this specific SAML relying party.

3. Step 2: Exporting PingFederate IdP Metadata

With the SP Connection fully configured on the PingFederate side, the next step was to give the Cascade application everything it needs to trust and validate assertions coming from PingFederate. This is done by

exporting PingFederate's IdP metadata - an XML document describing PingFederate's identity, its SSO/SLO endpoints, and its public signing certificate - from System → Protocol Metadata → Metadata Export.

The screenshot shows the PingFederate web interface. The top navigation bar includes 'AUTHENTICATION', 'APPLICATIONS', 'SECURITY', and 'SYSTEM'. The left sidebar shows 'Protocol Metadata' expanded, with 'Metadata Export' selected. The main content area is titled 'Metadata Export' and has tabs for 'Metadata Role', 'Metadata Mode', 'Connection Metadata', 'Metadata Signing', and 'Export & Summary'. The 'Metadata Role' tab is active, showing a question: 'This system is configured to act as both an IdP and an SP. For which role would you like to generate metadata?'. There are two radio button options: 'I AM THE IDENTITY PROVIDER (IDP)' (selected) and 'I AM THE SERVICE PROVIDER (SP)'. At the bottom right are 'Cancel' and 'Next' buttons.

Figure 35. Metadata Export: Metadata Role set to I am the Identity Provider (IdP).

The screenshot shows the 'Metadata Mode' tab selected. It contains the text: 'You can generate metadata specific to a connection, including the Attribute Contract and public key. Or you can provide a new contract and select a key manually. The resulting metadata may be shared with your partner to simplify connection creation.' There are two radio button options: 'USE A CONNECTION FOR METADATA GENERATION' (selected) and 'SELECT INFORMATION TO INCLUDE IN METADATA MANUALLY'. Below these is a checkbox for 'USE THE SECONDARY PORT FOR SOAP CHANNEL'. At the bottom right are 'Cancel', 'Previous', and 'Next' buttons.

Figure 36. Metadata Mode: Use a connection for metadata generation.

The screenshot shows the 'Connection Metadata' tab selected. It prompts the user to 'Select a connection that contains the Attribute Contract and Digital Signature Key you wish to include in the metadata.' A dropdown menu shows 'Cascade-Internal-Portal' selected. Below this, the 'Attribute Contract' is listed with fields: 'departmentNumber', 'givenName', 'mail', 'SAML_SUBJECT', and 'uid'. The 'DIGITAL SIGNATURE KEY' is shown as 'CN=PingFederate Signing Certificate, O=CyberCorp, C=US (01:A0:28:54:13:3A | Exp: Aug 22, 2027)'. The 'XML ENCRYPTION KEY' is noted as 'No XML key available for this connection'. At the bottom right are 'Cancel', 'Previous', and 'Next' buttons.

Figure 37. Connection Metadata: Cascade-Internal-Portal selected, showing its attribute contract and digital signature key.

PingFederate AUTHENTICATION APPLICATIONS SECURITY **SYSTEM**

Metadata Export

Metadata Role Metadata Mode Connection Metadata **Metadata Signing** Export & Summary

From this list of certificates, choose which one to use for signing the selected file.

SIGNING CERTIFICATE CN=PingFederate Signing Certificate... (01:A0:28:54:13:3A | Exp: Aug 22, 2027) ▼

☒ INCLUDE THIS CERTIFICATE'S PUBLIC KEY CERTIFICATE IN THE <KEYINFO> ELEMENT.

☐ INCLUDE THE RAW KEY IN THE SIGNATURE <KEYVALUE> ELEMENT.

SIGNING ALGORITHM RSA SHA256 ▼

Manage Certificates

Cancel Previous **Next**

Figure 38. Metadata Signing: PingFederate signing certificate, RSA SHA256.

PingFederate AUTHENTICATION APPLICATIONS SECURITY **SYSTEM**

Metadata Export

Metadata Role Metadata Mode Connection Metadata Metadata Signing **Export & Summary**

Click the Export button to export this metadata to the file system.

Metadata Export

Metadata Role	
Metadata role	Identity Provider

Metadata Mode	
Metadata mode	Use connection
Use the secondary port for SOAP channel	false

Connection Metadata	
Selected connection	Cascade-Internal-Portal
Attribute	SAML_SUBJECT
Attribute	uid
Attribute	mail
Attribute	givenName
Attribute	departmentNumber
Attribute	username
Digital Signature Key	CN=PingFederate Signing Certificate, O=CyberCorp, C=US (01:A0:28:54:13:3A Exp: Aug 22, 2027)

Metadata Signing	
Signing Certificate	CN=PingFederate Signing Certificate, O=CyberCorp, C=US (01:A0:28:54:13:3A Exp: Aug 22, 2027)
Include Certificate in KeyInfo	true
Include Raw Key in Key/Value	false
Selected Signing Algorithm	RSA SHA256

Export

Cancel Previous **Done**

Figure 39. Export & Summary: the full metadata export configuration before exporting.

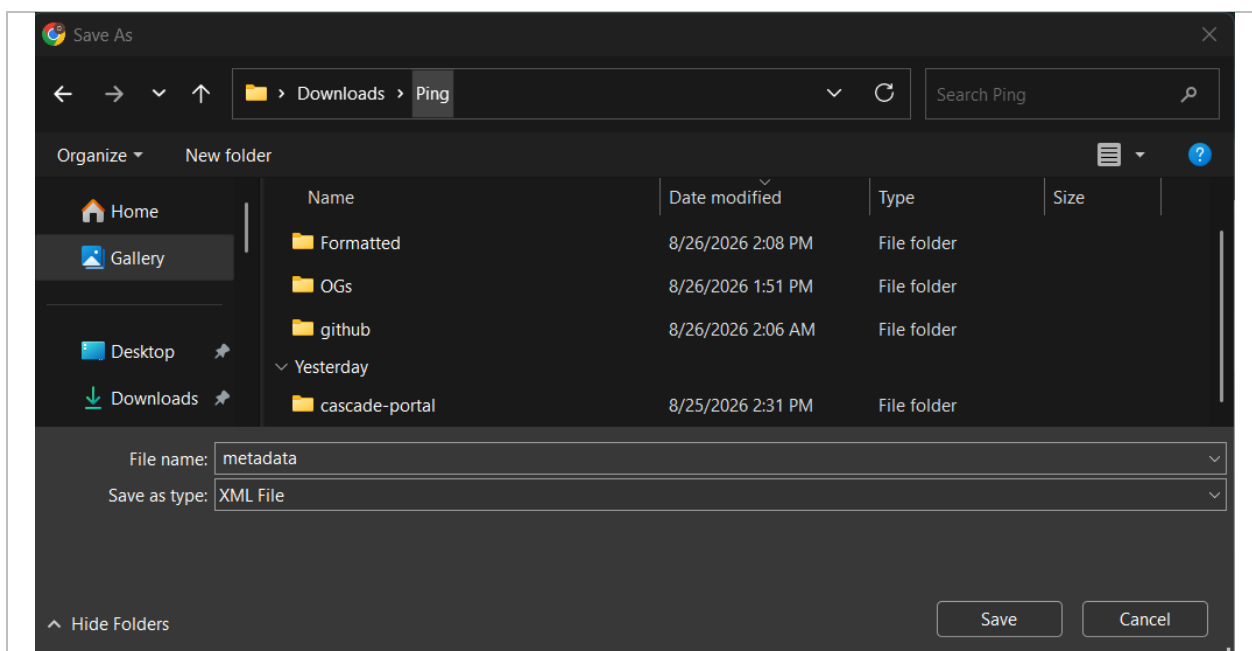


Figure 40. Save As dialog - saving the exported metadata as metadata.xml.

What we did and why

We generated metadata for the Identity Provider (IdP) role, since PingFederate is acting as the IdP in this relationship (Cascade is the SP). For Metadata Mode we chose Use a connection for metadata generation and selected the Cascade-Internal-Portal connection, rather than manually selecting which information to include. Scoping the export to a specific connection means the exported metadata reflects exactly the attribute contract already built for Cascade - departmentNumber, givenName, mail, SAML_SUBJECT, uid, and username - plus the PingFederate signing certificate tied to that connection, rather than a generic, connection-agnostic metadata document.

On Metadata Signing, the metadata file itself was signed using the same PingFederate signing certificate (RSA SHA256, certificate included in KeyInfo). Signing the metadata document is a separate step from signing individual SAML assertions - it protects the metadata file itself from tampering while it's transferred out of PingFederate and into the Cascade application's configuration.

The Export & Summary screen showed the complete picture before export: IdP role, connection-scoped mode, the Cascade-Internal-Portal connection, its attribute list, and both the connection's digital signature key and the metadata's own signing certificate. Clicking Export prompted a Save As dialog, and the file was saved as metadata.xml. This file is what would ultimately supply a SAML library with PingFederate's SSO endpoint URL and public certificate - though, as Step 3 shows, the equivalent values were configured directly as environment variables and a certificate file in this lab, rather than by importing metadata.xml wholesale into the app.

4. Step 3: Configuring the Cascade Portal for SAML

With PingFederate fully configured, the remaining work moved to the Cascade application server itself: wiring its SAML library to PingFederate, restyling the PingFederate-hosted login page, and then diagnosing two real configuration errors before authentication finally worked end to end.

4.1 Backing Up and Updating Application Configuration (.env)

```

appserver@appserver:~/cascade-internal-portal/cascade-portal$ cp .env .env.backup-$(date +%Y%m%d-%H%M%S)
appserver@appserver:~/cascade-internal-portal/cascade-portal$ ls -al
total 164
drwxrwxr-x 11 appserver appserver 4096 Aug 26 16:33 .
drwxrwxr-x 3 appserver appserver 4096 Aug 21 23:30 ..
drwxrwxr-x 2 appserver appserver 4096 Aug 22 00:23 certs
drwxrwxr-x 2 appserver appserver 4096 Aug 21 23:30 config
drwxrwxr-x 2 appserver appserver 4096 Aug 21 23:53 data
-rw-rw-r-- 1 appserver appserver 2994 Aug 22 01:25 .env
-rw-rw-r-- 1 appserver appserver 2994 Aug 26 16:33 .env.backup-20260826-163304
-rw-rw-r-- 1 appserver appserver 2935 Aug 21 23:29 .env.example
-rw-rw-r-- 1 appserver appserver 50 Aug 21 23:29 .gitignore
drwxrwxr-x 2 appserver appserver 4096 Aug 22 01:00 middleware
drwxrwxr-x 166 appserver appserver 4096 Aug 21 23:51 node_modules
-rw-rw-r-- 1 appserver appserver 888 Aug 21 23:29 package.json
-rw-rw-r-- 1 appserver appserver 85468 Aug 21 23:51 package-lock.json
drwxrwxr-x 3 appserver appserver 4096 Aug 21 23:30 public
-rw-rw-r-- 1 appserver appserver 10993 Aug 21 23:29 README.md
drwxrwxr-x 2 appserver appserver 4096 Aug 22 01:55 routes
drwxrwxr-x 2 appserver appserver 4096 Aug 21 23:30 scripts
-rw-rw-r-- 1 appserver appserver 2955 Aug 21 23:29 server.js
drwxrwxr-x 3 appserver appserver 4096 Aug 21 23:30 views

```

Figure 41. Backing up .env and listing the Cascade application directory.

```

GNU nano 7.2 .env *
# --- App ---
PORT=3000
APP_BASE_URL=https://192.168.131.160:3000
SESSION_SECRET=change-this-to-a-long-random-string
NODE_ENV=development
# Set to "true" once you're running behind real TLS (see README "Going to H
TRUST_PROXY=false

# --- Which SSO protocol to use for the "Log in" button ---
# One of: saml | oidc
DEFAULT_PROTOCOL=saml

# --- SAML (PingFederate as IdP) ---
# PingFederate IdP metadata / SSO endpoint, e.g. https://localhost:9031/idp>
SAML_IDP_SSO_URL=https://192.168.131.159:9031/idp/SSO.saml2
# PingFederate SLO endpoint, e.g. https://localhost:9031/idp/SLO.saml2
SAML_IDP_SLO_URL=https://192.168.131.159:9031/idp/SLO.saml2
# Entity ID you configured for PingFederate's IdP role (Server Settings > F>
SAML_IDP_ISSUER=evaluation
# Entity ID you gave THIS app when creating the SP Connection in PingFedera>
SAML_SP_ISSUER=urn:cascade:internal-portal
# Path to the PingFederate signing certificate you exported (see README)
SAML_IDP_CERT_PATH=./certs/pingfederate-idp-signing.pem
# This app's own keypair, used to sign SP-initiated AuthnRequests (optional>
SAML_SP_PRIVATE_KEY_PATH=./certs/sp-private-key.pem
SAML_SP_CERT_PATH=./certs/sp-cert.pem
SAML_CALLBACK_PATH=/auth/saml/callback

```

Figure 42. Editing .env to add the SAML configuration values.

Note: This lab's screenshots include internal IP addresses (192.168.131.159/.160), file paths, and a signing certificate fingerprint. These are specific to this throwaway Docker lab environment, but as with Labs 1 and 2, it's worth treating anything that looks like a real endpoint or credential as something to review before publishing to a public repository.

What we did and why

Configuration work moved from PingFederate to the Cascade application server itself. Before changing anything, the existing `.env` file was copied to a timestamped backup (`.env.backup-20260826-163304`) so the working configuration could be restored if the SAML changes broke something - a basic but important change-management habit when editing a live application's configuration.

The `.env` file was then edited to add the SAML-specific settings the app's SAML strategy (`config/saml.js`, see 4.2) reads at startup:

- `DEFAULT_PROTOCOL=saml` - tells the app's login page to default to the SAML flow (the app also supports OIDC, as seen in the final proof-of-authentication screenshots)
- `SAML_IDP_SSO_URL=https://192.168.131.159:9031/idp/SSO.saml2` - PingFederate's SAML 2.0 SSO endpoint, where the app sends its AuthnRequest
- `SAML_IDP_SLO_URL=https://192.168.131.159:9031/idp/SLO.saml2` - PingFederate's logout endpoint, configured for completeness even though SLO wasn't enabled on the PingFederate side in this lab
- `SAML_IDP_ISSUER=evaluation` - the entity ID PingFederate uses for its own IdP role, taken from PingFederate's Server Settings
- `SAML_SP_ISSUER=urn:cascade:internal-portal` - the entity ID this app declares as itself when it sends an AuthnRequest
- `SAML_IDP_CERT_PATH=./certs/pingfederate-idp-signing.pem` - a local copy of PingFederate's public signing certificate, used to verify incoming assertions are genuinely signed by PingFederate
- `SAML_SP_PRIVATE_KEY_PATH / SAML_SP_CERT_PATH` - this app's own optional keypair, usable if it ever needs to sign its own outgoing AuthnRequests
- `SAML_CALLBACK_PATH=/auth/saml/callback` - the route that, combined with `APP_BASE_URL`, forms the exact ACS URL configured back in PingFederate (see 2.7)

At this point, `SAML_SP_ISSUER` was set to `urn:cascade:internal-portal` - which, as 4.5 shows, did not yet match the Partner's Entity ID (`CascadeInternalPortal`) configured in PingFederate back in 2.2. This mismatch was still latent here and wouldn't surface until the app actually attempted a login.

4.2 Fixing the SAML NameID Format in Code

```
return new SamlStrategy(
{
  // Where PingFederate expects the AuthnRequest
  entryPoint: process.env.SAML_IDP_SSO_URL,
  logoutUrl: process.env.SAML_IDP_SLO_URL,
  // This app's own identity in the assertion audience
  issuer: process.env.SAML_SP_ISSUER,
  callbackUrl: `${process.env.APP_BASE_URL}${process.env.SAML_CALLBACK_PATH}`,
  cert: idpCert, // PingFederate's signing cert - validates the assertion came from Ping
  privateKey: spPrivateKey, // optional: signs our outgoing AuthnRequest
  decryptionPvk: spPrivateKey, // optional: decrypts assertions if you enable encryption in PF
  signatureAlgorithm: 'sha256',
  wantAssertionsSigned: true,
  identifierFormat: 'urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified',
  disableRequestedAuthnContext: true
},
// verify callback (async form)
(profile, done) => {
  const user = {
    protocol: 'saml'
  }
}
```

Figure 43. config/saml.js - confirming identifierFormat matches PingFederate's unspecified NameID format.

What we did and why

config/saml.js is where the Node.js application builds its SAML strategy object from the environment variables set in 4.1. Reviewing this file, identifierFormat was set to urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified - deliberately matching the Subject Name Format left at its default in the PingFederate Attribute Contract (see 2.5). If these two values didn't match, the SP and IdP would disagree on how the NameID in the assertion should be interpreted, which can cause a SAML library to reject an otherwise valid assertion.

The rest of this configuration block is worth understanding line by line, since it's the code-level implementation of the trust relationship configured in PingFederate:

- entryPoint / logoutUrl read from SAML_IDP_SSO_URL / SAML_IDP_SLO_URL - where PingFederate expects the AuthnRequest
- issuer reads from SAML_SP_ISSUER - this app's own identity in the assertion audience
- callbackUrl is built from APP_BASE_URL and SAML_CALLBACK_PATH - must match the ACS URL in PingFederate exactly
- cert: idpCert - PingFederate's signing certificate, used to validate the assertion actually came from Ping and wasn't forged
- privateKey / decryptionPvk - this app's own key, for optionally signing outgoing requests or decrypting encrypted assertions (not currently used, since PingFederate's Encryption Policy is None)
- wantAssertionsSigned: true - tells the SAML library to reject any assertion that isn't signed, matching PingFederate's Always Sign Assertion setting
- disableRequestedAuthnContext: true - tells the app not to request a specific authentication context class from PingFederate, letting PingFederate's own authentication policy (from Lab 2) decide how the user is authenticated, rather than the SP dictating it

4.3 Verifying Configuration and Connectivity

```

appserver@appserver:~/cascade-internal-portal/cascade-portal$ grep -E 'APP_BASE_URL|SAML_IDP_SSO_URL|SAML_IDP_SLO_URL|SAML_IDP_ISSUER|SAML_SP_ISSUER|SAML_IDP_CERT_PATH|SAML_CALLBACK_PATH' .env
APP_BASE_URL=https://192.168.131.160:3000
SAML_IDP_SSO_URL=https://192.168.131.159:9031/idp/SSO.saml2
SAML_IDP_SLO_URL=https://192.168.131.159:9031/idp/SLO.saml2
SAML_IDP_ISSUER=evaluation
SAML_SP_ISSUER=urn:cascade:internal-portal
SAML_IDP_CERT_PATH=./certs/pingfederate-idp-signing.pem
SAML_CALLBACK_PATH=/auth/saml/callback
appserver@appserver:~/cascade-internal-portal/cascade-portal$ identifierFormat: 'urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified',
identifierFormat:: command not found
appserver@appserver:~/cascade-internal-portal/cascade-portal$ grep -n "identifierFormat" config/saml.js
37:     identifierFormat: 'urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified'
'
appserver@appserver:~/cascade-internal-portal/cascade-portal$ openssl x509 -in certs/pingfederate-idp-signing.pem -noout -subject -issuer -dates
subject=C = US, O = CyberCorp, CN = PingFederate Signing Certificate
issuer=C = US, O = CyberCorp, CN = PingFederate Signing Certificate
notBefore=Aug 22 07:16:33 2026 GMT
notAfter=Aug 22 07:16:33 2027 GMT
appserver@appserver:~/cascade-internal-portal/cascade-portal$ |

```

Figure 44. Verifying the SAML environment variables and the PingFederate signing certificate with openssl x509.

```

appserver@appserver:~/cascade-internal-portal/cascade-portal$ curl -k -I https://192.168.131.159:9031
HTTP/2 403
date: Wed, 26 Aug 2026 23:48:53 GMT
x-frame-options: SAMEORIGIN
referrer-policy: origin
x-robots-tag: noindex
content-type: text/html; charset=utf-8
cache-control: must-revalidate, no-cache, no-store
set-cookie: PF=kr2QAGpomP1AjZ9cvoJY2E; Path=/; Secure; HttpOnly
expires: Thu, 01 Jan 1970 00:00:00 GMT
content-length: 1485

appserver@appserver:~/cascade-internal-portal/cascade-portal$ curl -k -I https://192.168.131.159:9031/idp/SSO.saml2
HTTP/2 400
date: Wed, 26 Aug 2026 23:49:06 GMT
x-frame-options: SAMEORIGIN
referrer-policy: origin
x-robots-tag: noindex
pragma: no-cache
set-cookie: PF=cZnTvov8Jg7Ne04Qh2PpFG; Path=/; Secure; HttpOnly; SameSite=None
content-type: text/html; charset=utf-8
cache-control: must-revalidate, no-cache, no-store
content-length: 1639

```

Figure 45. Confirming basic connectivity to PingFederate with curl.

What we did and why

Before attempting an actual login, the configuration was verified from the command line. A `grep` across `.env` confirmed every SAML-related value was set as intended (`APP_BASE_URL`, both `PingFederate` URLs, both issuers, the cert path, and the callback path). An `openssl x509` check against the locally stored `pingfederate-idp-signing.pem` confirmed the certificate's subject and issuer both read `C=US, O=CyberCorp, CN=PingFederate Signing Certificate` - confirming this is a self-signed certificate genuinely belonging to the `PingFederate` instance, valid from August 22, 2026 through August 22, 2027, matching the certificate selected inside `PingFederate` itself (see 2.8).

Two `curl -k -I` checks confirmed basic network reachability to `PingFederate`: a request to the base `PingFederate` host returned HTTP 403 (expected - there's no resource at that exact path, but the server responded), and a request to the actual SSO endpoint, `/idp/SSO.saml2`, returned HTTP 400 (also expected, since a bare GET with no `SAMLRequest` parameter is an invalid request - the important result is that `PingFederate` is reachable and responding at all, not the specific status code). These checks isolate network and TLS trust issues from the SAML logic itself, so if a later login attempt failed, connectivity could already be ruled out as a cause.

4.4 Restyling the PingFederate Login Page

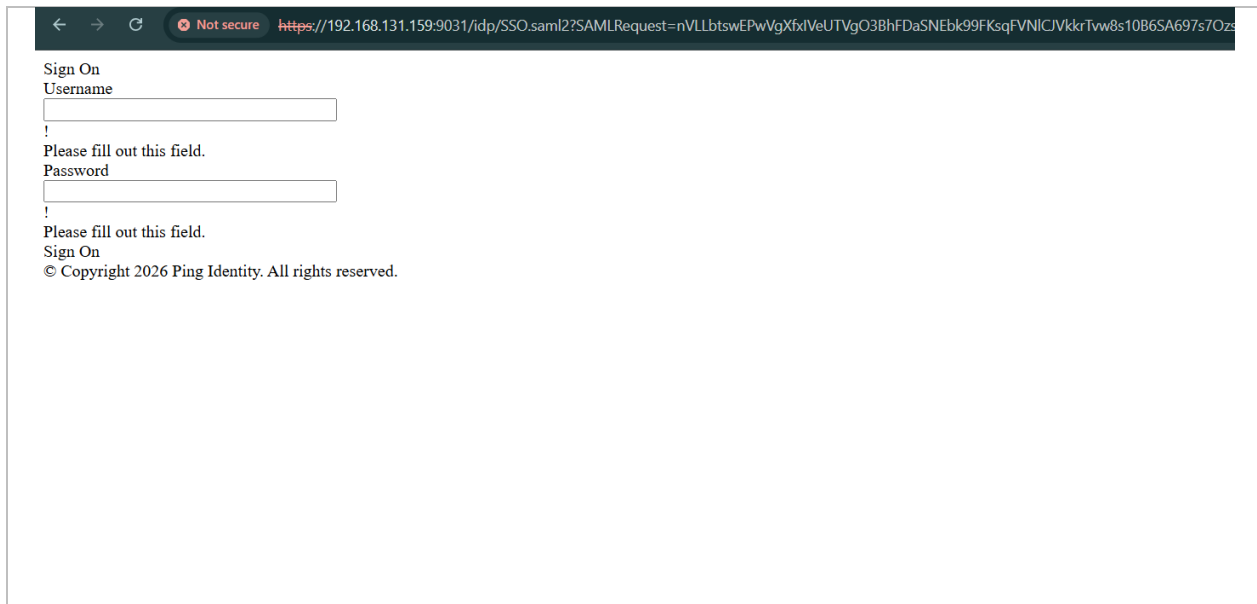


Figure 46. The default, unstyled `PingFederate` login form before restyling.



Figure 47. Locating `html.form.login.template.html` inside the running `PingFederate` container.

```
devanshu@ping-lab:~$ docker exec pingfederate cp \
/opt/out/instance/server/default/conf/template/html.form.login.template.html \
/opt/out/instance/server/default/conf/template/html.form.login.template.html.bak
devanshu@ping-lab:~$
```

Figure 48. Backing up the login template before editing it.

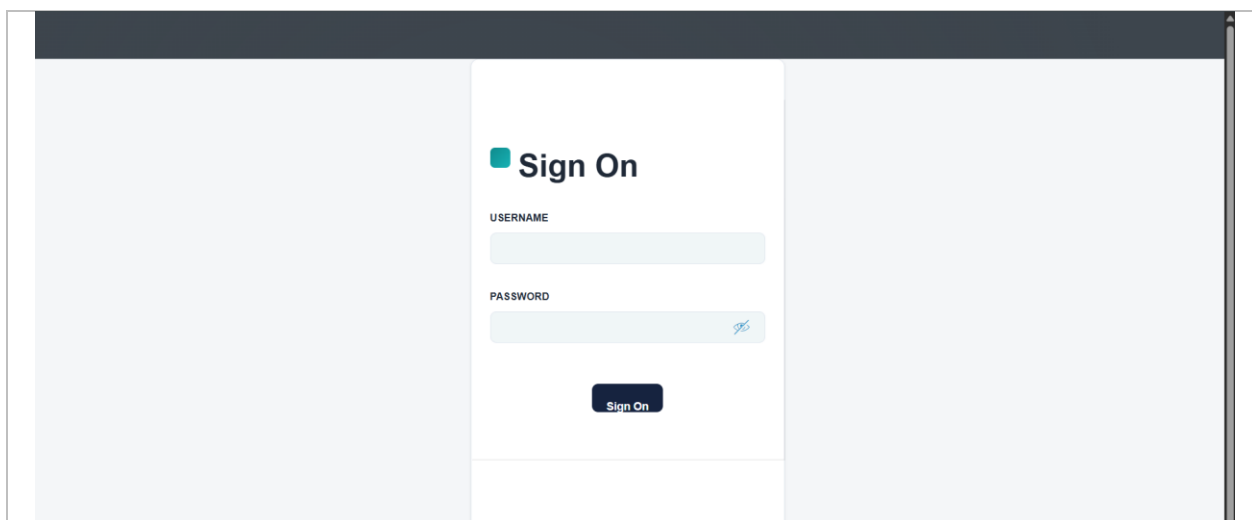


Figure 49. The restyled PingFederate login form after the new template was pushed.

What we did and why

With the application side wired up, attention turned to the PingFederate login page itself - the HTML form a user's browser lands on after being redirected by Cascade's AuthnRequest. By default this page is bare, unstyled HTML: a plain Sign On heading, two unstyled fields, and a copyright line, with no styling applied at all.

The template responsible for this page was located inside the running PingFederate container with `docker exec ... find`, which located it at `/opt/out/instance/server/default/conf/template/html.form.login.template.html`.

Before editing it, a backup copy was made in place (`html.form.login.template.html.bak`) using `docker exec ... cp` - the same change-management habit applied to the `.env` file in 4.1, so the original template could be restored if the new one broke the login flow.

A new version of the template was then written and pushed to that same location, applying real styling: a teal accent square, a proper Sign On heading, labeled USERNAME and PASSWORD fields with a password-visibility toggle, and a styled Sign On button. Because this file lives inside PingFederate itself (not the Cascade application), this change affects the login experience for every SP connection that uses this adapter's login template - it's PingFederate's own IdP-side branding, not something Cascade controls.

4.5 Troubleshooting: Error 1 - Unknown SP Entity ID

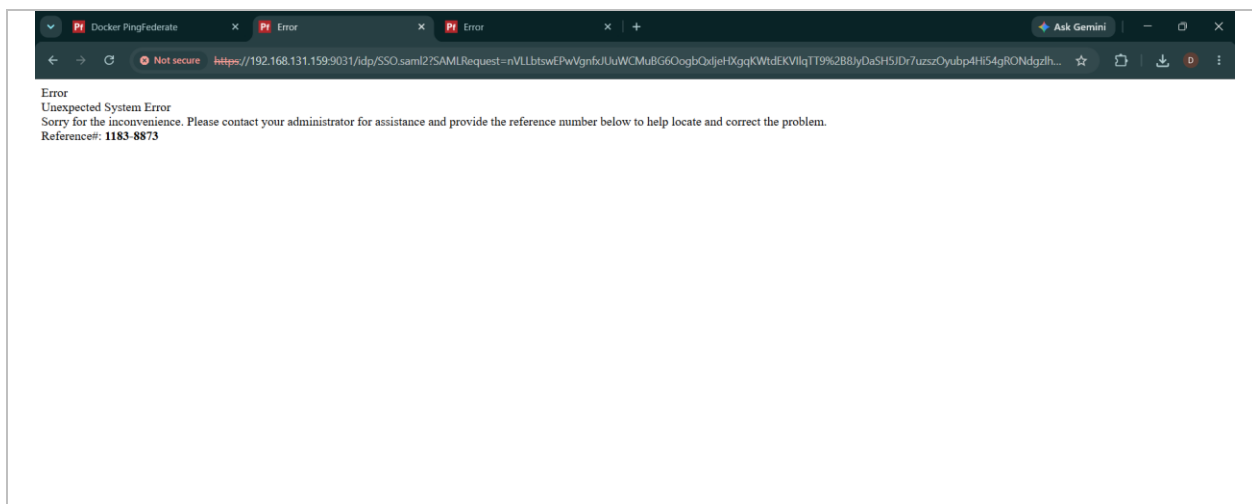


Figure 50. Error 1: “Unexpected System Error,” reference #1183-8873.

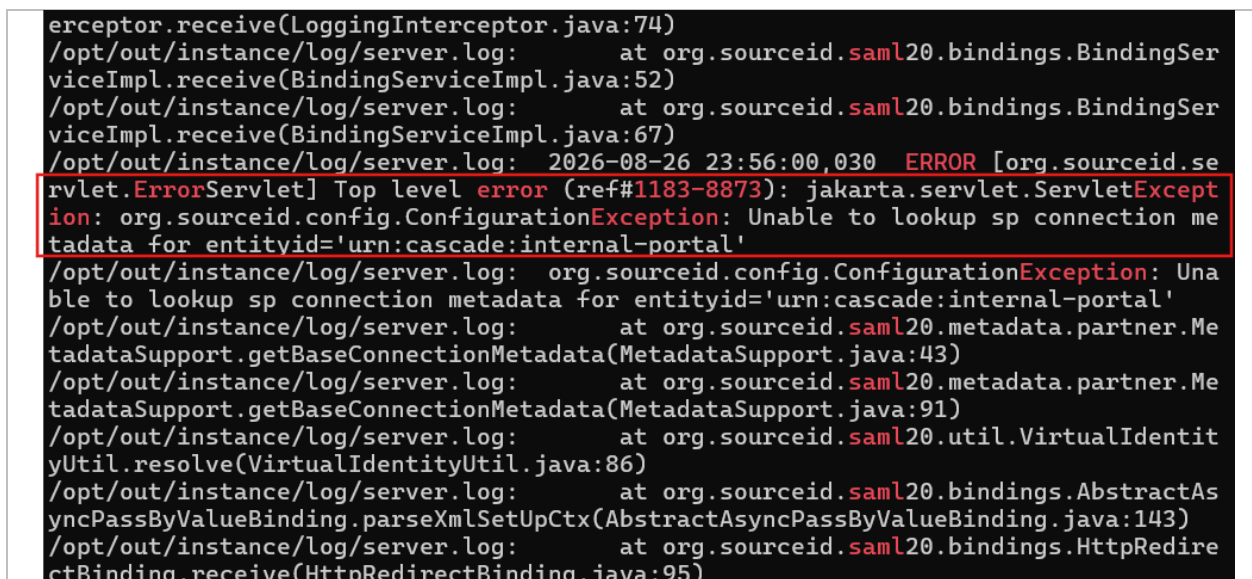
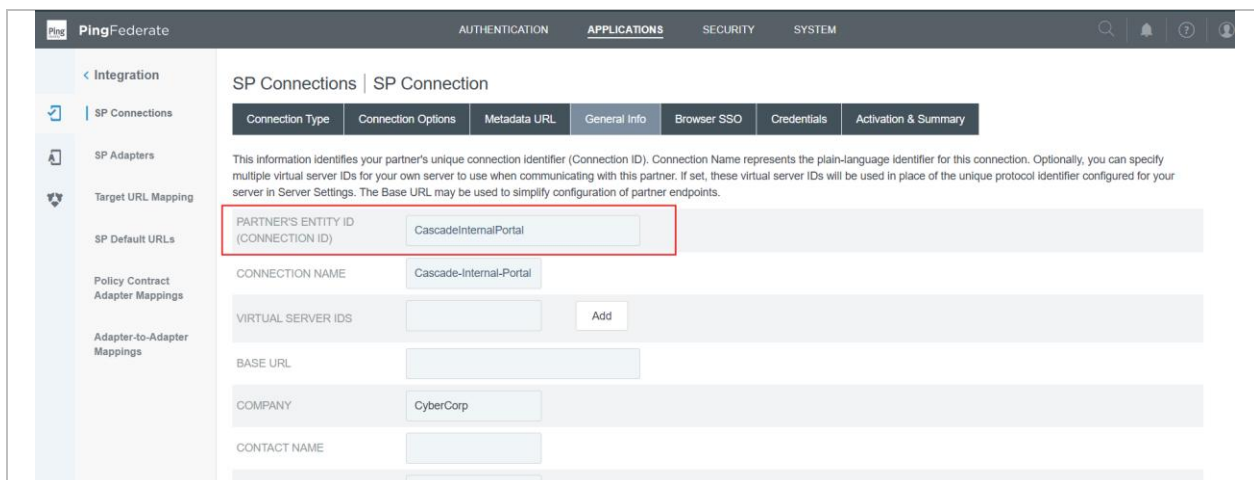


Figure 51. Server log for reference #1183-8873: ConfigurationException - unable to look up SP connection metadata for entity ID urn:cascade:internal-portal.



The screenshot shows the PingFederate interface with the 'SP Connections' tab selected. The 'General Info' sub-tab is active. The 'PARTNER'S ENTITY ID (CONNECTION ID)' field contains the value 'CascadeInternalPortal', which is highlighted with a red rectangle. Other fields include 'CONNECTION NAME' (Cascade-Internal-Portal), 'VIRTUAL SERVER IDS' (empty), 'BASE URL' (empty), 'COMPANY' (CyberCorp), and 'CONTACT NAME' (empty).

PingFederate AUTHENTICATION APPLICATIONS SECURITY SYSTEM

< Integration

SP Connections

SP Adapters

Target URL Mapping

SP Default URLs

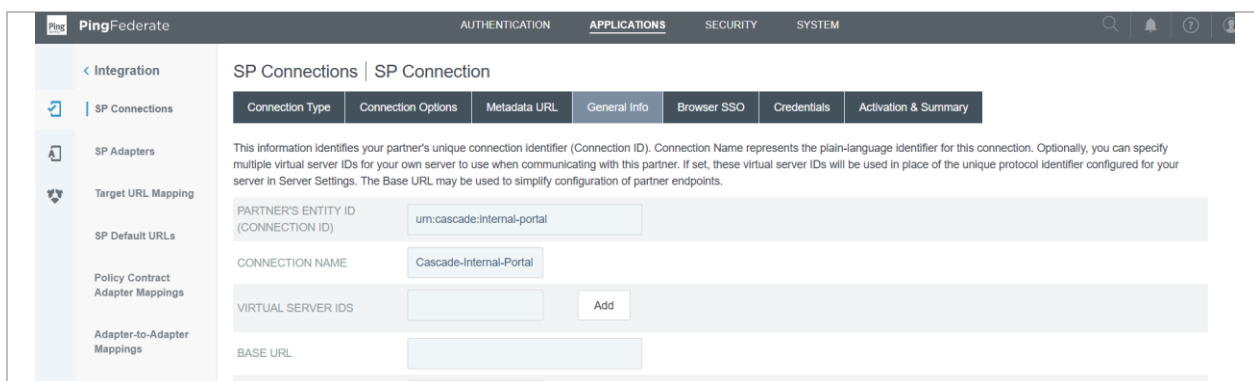
Policy Contract Adapter Mappings

Adapter-to-Adapter Mappings

SP Connections | SP Connection

Connection Type	Connection Options	Metadata URL	General Info	Browser SSO	Credentials	Activation & Summary
This information identifies your partner's unique connection identifier (Connection ID). Connection Name represents the plain-language identifier for this connection. Optionally, you can specify multiple virtual server IDs for your own server to use when communicating with this partner. If set, these virtual server IDs will be used in place of the unique protocol identifier configured for your server in Server Settings. The Base URL may be used to simplify configuration of partner endpoints.						
PARTNER'S ENTITY ID (CONNECTION ID)		CascadeInternalPortal				
CONNECTION NAME		Cascade-Internal-Portal				
VIRTUAL SERVER IDS		Add				
BASE URL						
COMPANY		CyberCorp				
CONTACT NAME						

Figure 52. Rechecking the SP Connection's General Info - Partner's Entity ID was `CascadeInternalPortal`, not `urn:cascade:internal-portal`.



The screenshot shows the same PingFederate interface as Figure 52, but the 'PARTNER'S ENTITY ID (CONNECTION ID)' field now contains the corrected value 'urn:cascade:internal-portal'.

PingFederate AUTHENTICATION APPLICATIONS SECURITY SYSTEM

< Integration

SP Connections

SP Adapters

Target URL Mapping

SP Default URLs

Policy Contract Adapter Mappings

Adapter-to-Adapter Mappings

SP Connections | SP Connection

Connection Type	Connection Options	Metadata URL	General Info	Browser SSO	Credentials	Activation & Summary
This information identifies your partner's unique connection identifier (Connection ID). Connection Name represents the plain-language identifier for this connection. Optionally, you can specify multiple virtual server IDs for your own server to use when communicating with this partner. If set, these virtual server IDs will be used in place of the unique protocol identifier configured for your server in Server Settings. The Base URL may be used to simplify configuration of partner endpoints.						
PARTNER'S ENTITY ID (CONNECTION ID)		urn:cascade:internal-portal				
CONNECTION NAME		Cascade-Internal-Portal				
VIRTUAL SERVER IDS		Add				
BASE URL						

Figure 53. Solution: Partner's Entity ID corrected to `urn:cascade:internal-portal`.

What we did and why

The first end-to-end login attempt failed immediately with a generic “Unexpected System Error” page and reference number 1183-8873. That reference number is the key to diagnosing the real problem - PingFederate logs a matching entry for every reference number it generates.

Searching PingFederate's `server.log` for that reference number surfaced the actual exception: `org.sourceid.config.ConfigurationException: Unable to lookup sp connection metadata for entityid='urn:cascade:internal-portal'`. In other words: PingFederate received an `AuthnRequest` whose issuer was `urn:cascade:internal-portal`, tried to look up an SP connection with that exact entity ID, and found nothing - because the SP Connection created in 2.2 was registered under the Partner's Entity ID `CascadeInternalPortal`, not `urn:cascade:internal-portal`. This is precisely the mismatch introduced back in 4.1: the app's `SAML_SP_ISSUER` environment variable didn't match what PingFederate had on file for this connection.

The fix was to return to the SP Connection's General Info tab in PingFederate and change the Partner's Entity ID from `CascadeInternalPortal` to `urn:cascade:internal-portal`, so it exactly matches the issuer value the application actually sends. (The alternative fix would have been to change the app's `SAML_SP_ISSUER` to `CascadeInternalPortal` instead - either side can be changed, since what matters is that both sides agree on the same value. Editing the PingFederate side was simpler here because it didn't require redeploying the application.) This is a foundational SAML troubleshooting lesson: the SP's entity ID, as declared in its `AuthnRequest` issuer, must be byte-for-byte identical to the Partner's Entity ID configured on the IdP side, or the IdP has no way to know which connection's trust configuration to apply.

4.6 Troubleshooting: Error 2 - No Authentication Methods Available

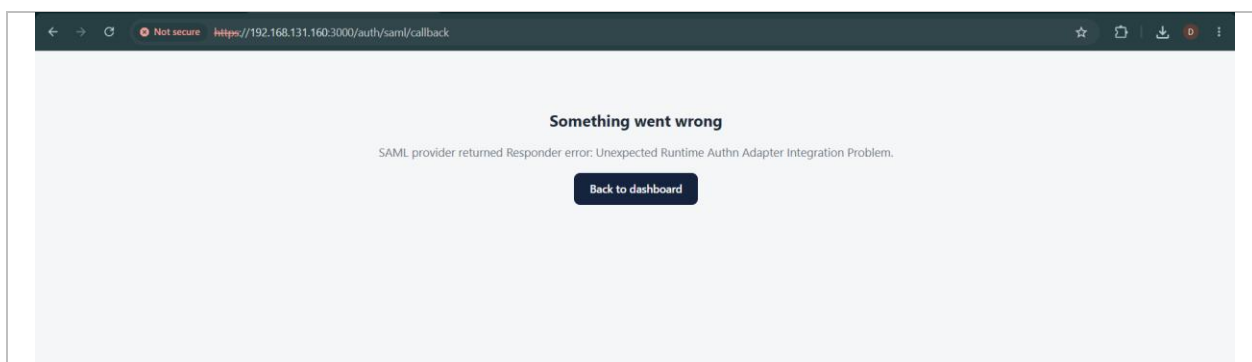


Figure 54. Error 2: “Something went wrong - SAML provider returned Responder error: Unexpected Runtime Authn Adapter Integration Problem.”

```

devanshu@ping-lab:~$ docker logs --tail 300 pingfederate 2>&1 | grep -i -E "adapter|authn|authentication|exception|error"
/opt/out/instance/log/server.log: 2026-08-27 08:06:57,303 tid:95051K08_TcNkgqkx9UoF4LF-4 ERROR [org.sourceid.websso.profiles.ExceptionLoggingSupport] Exception occurred
during request processing
org.sourceid.websso.profiles.RequestProcessingException: Unexpected Runtime Authn Adapter Integration Problem.
at org.sourceid.saml20.profiles.idp.HandleAuthnRequest.handle(HandleAuthnRequest.java:227)
at org.sourceid.servlet.filter.SimultaneousAuthnRequestClearingFilter.doFilter(SimultaneousAuthnRequestClearingFilter.java:58)
at org.sourceid.servlet.filter.SimultaneousAuthnRequestCheckingFilter.doFilter(SimultaneousAuthnRequestCheckingFilter.java:148)
Caused by: org.sourceid.saml20.adapter.AuthnAdapterException$NoMappedAdapters: No authentication methods available.
at org.sourceid.saml20.profiles.idp.AuthnSourceSupportBase.doLegacySelection(AuthnSourceSupportBase.java:2814)
at org.sourceid.saml20.profiles.idp.AuthnSourceSupportBase.doLookupAuthNs(AuthnSourceSupportBase.java:1147)
at org.sourceid.saml20.profiles.idp.AuthnSourceSupportBase.lambda$lookupAuthNs$0(AuthnSourceSupportBase.java:1031)
at org.sourceid.websso.authn.ApiRedirectWrapper.wrap(ApiRedirectWrapper.java:239)
at org.sourceid.saml20.profiles.idp.AuthnSourceSupportBase.lookupAuthNs(AuthnSourceSupportBase.java:1031)
at org.sourceid.saml20.profiles.idp.HandleAuthnRequest.doResume(HandleAuthnRequest.java:282)
2026-08-27 08:07:31,340 tid:95051K08_TcNkgqkx9UoF4LF-4 ERROR [org.sourceid.websso.profiles.ExceptionLoggingSupport] Exception occurred
during request processing
org.sourceid.websso.profiles.RequestProcessingException: Unexpected Runtime Authn Adapter Integration Problem.
at org.sourceid.saml20.profiles.idp.HandleAuthnRequest.handle(HandleAuthnRequest.java:227)
at org.sourceid.servlet.filter.SimultaneousAuthnRequestClearingFilter.doFilter(SimultaneousAuthnRequestClearingFilter.java:58)
at org.sourceid.servlet.filter.SimultaneousAuthnRequestCheckingFilter.doFilter(SimultaneousAuthnRequestCheckingFilter.java:148)
Caused by: org.sourceid.saml20.adapter.AuthnAdapterException$NoMappedAdapters: No authentication methods available.
at org.sourceid.saml20.profiles.idp.AuthnSourceSupportBase.doLegacySelection(AuthnSourceSupportBase.java:2814)
at org.sourceid.saml20.profiles.idp.AuthnSourceSupportBase.doLookupAuthNs(AuthnSourceSupportBase.java:1147)
at org.sourceid.saml20.profiles.idp.AuthnSourceSupportBase.lambda$lookupAuthNs$0(AuthnSourceSupportBase.java:1031)
at org.sourceid.websso.authn.ApiRedirectWrapper.wrap(ApiRedirectWrapper.java:239)
at org.sourceid.saml20.profiles.idp.AuthnSourceSupportBase.lookupAuthNs(AuthnSourceSupportBase.java:1031)
at org.sourceid.saml20.profiles.idp.HandleAuthnRequest.doResume(HandleAuthnRequest.java:282)

```

Figure 55. PingFederate container logs: AuthnAdapterException\$NoMappedAdapters - no authentication methods available.

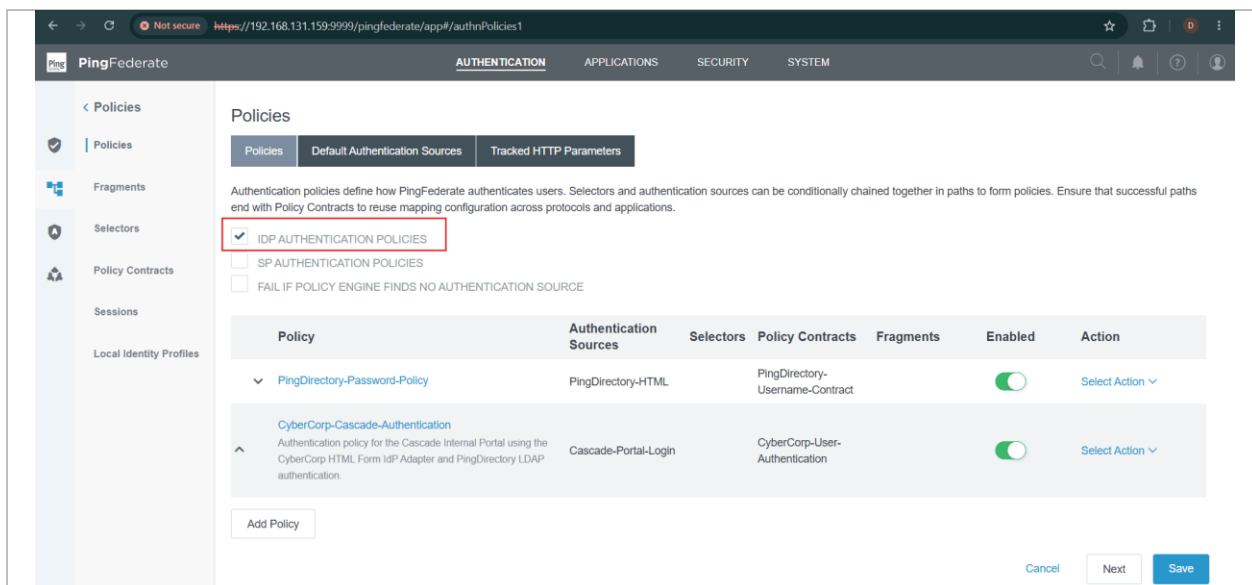


Figure 56. Solution: the IDP AUTHENTICATION POLICIES master switch on the Policies screen was unchecked.

What we did and why

With the entity ID corrected, the next login attempt got further - PingFederate recognized the connection and presumably showed a login form - but the callback to Cascade (<https://192.168.131.160:3000/auth/saml/callback>) landed on an app-side error page: "SAML provider returned Responder error: Unexpected Runtime Authn Adapter Integration Problem." A Responder error means PingFederate itself reported the failure back in the SAML response, rather than the fault being on Cascade's side.

Filtering PingFederate's own container logs for authentication-related errors (`docker logs ... | grep -i -E "adapter|authn|..."`) showed the real cause: `org.sourceid.saml20.adapter.AuthnAdapterException$NoMappedAdapters:` No authentication methods available, thrown from `HandleAuthnRequest` while trying to resolve which authentication source to use for this request. Despite the `CyberCorp-Cascade-Authentication` policy existing, being enabled, and correctly mapping `Cascade-Portal-Login` to `CyberCorp-User-Authentication` (all configured back in Lab 2), PingFederate wasn't considering it at all: the authentication policy existed and was configured, but it wasn't currently being selected or evaluated for this SP Connection's SSO request.

The fix was found on Authentication → Policies: the `IDP AUTHENTICATION POLICIES` checkbox at the top of the page - the master switch that tells PingFederate's policy engine to actually evaluate IdP authentication policies at all - was unchecked. With it unchecked, PingFederate's policy engine had no authentication sources to consider for any SP-initiated request, regardless of how well an individual policy was configured underneath it. Checking `IDP AUTHENTICATION POLICIES` and saving resolved the error. This is a good reminder that PingFederate's authentication policy engine has both a global enable/disable switch and per-policy configuration - both have to be correct, and it's easy to build a perfectly valid policy that's simply never consulted because the engine-level switch was never turned on.

4.7 Proof of Authentication

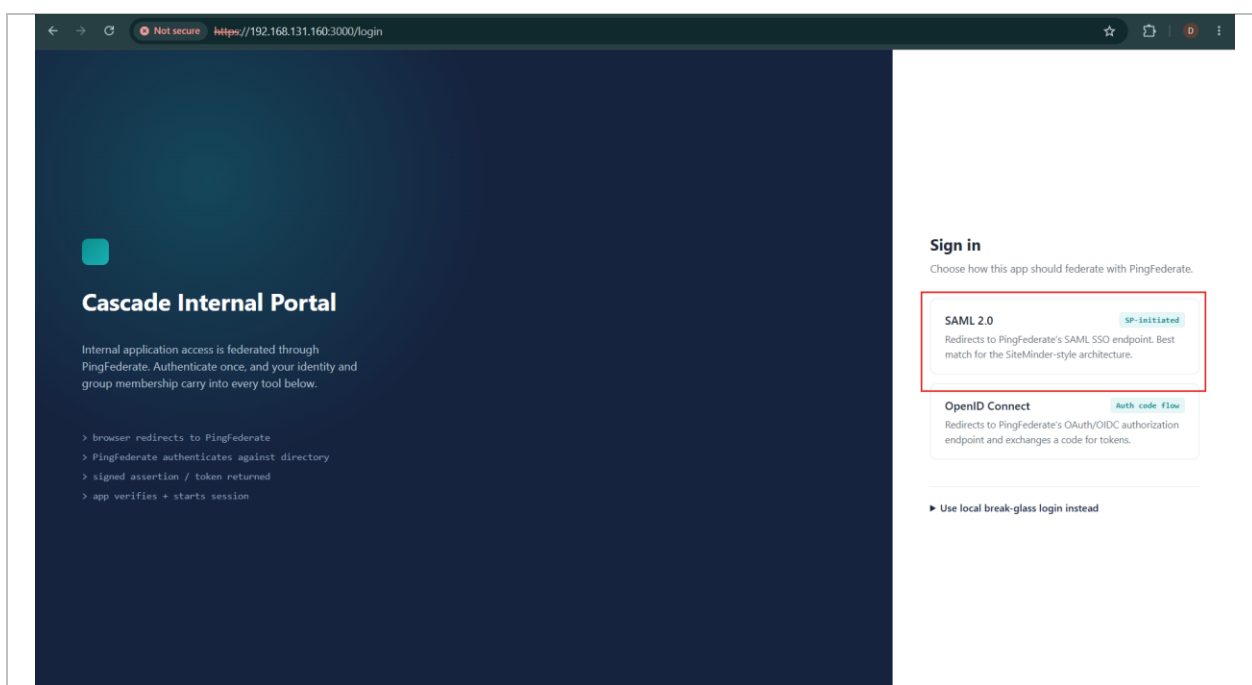


Figure 57. The Cascade Internal Portal sign-in page, offering SAML 2.0 (SP-initiated) or OpenID Connect.

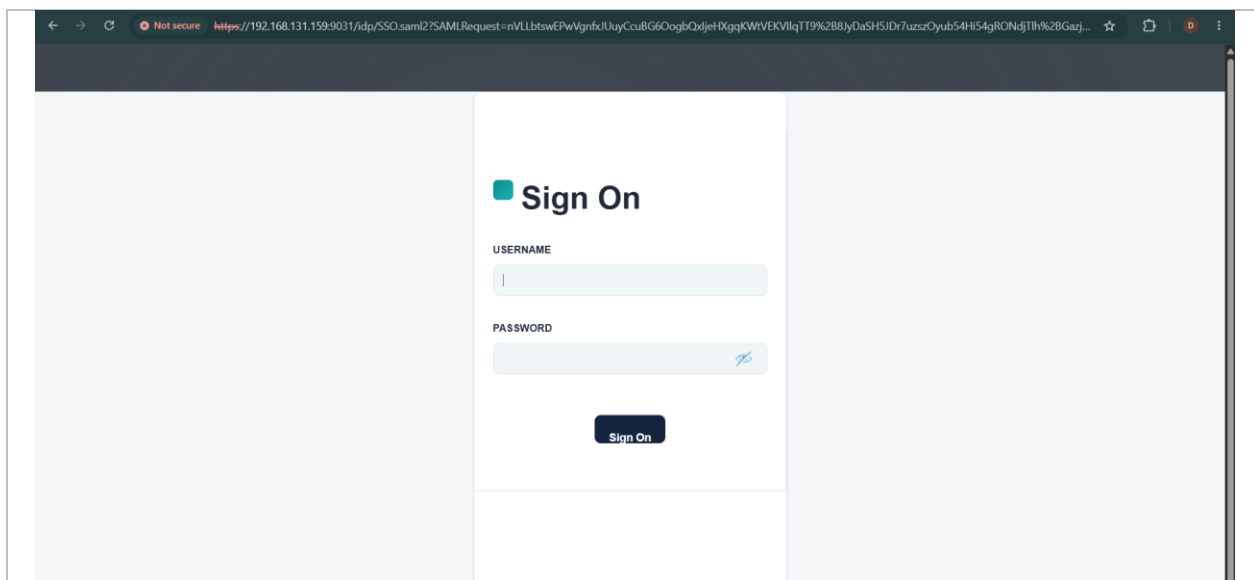


Figure 58. Redirected to PingFederate's SSO.saml2 endpoint, showing the restyled login form.

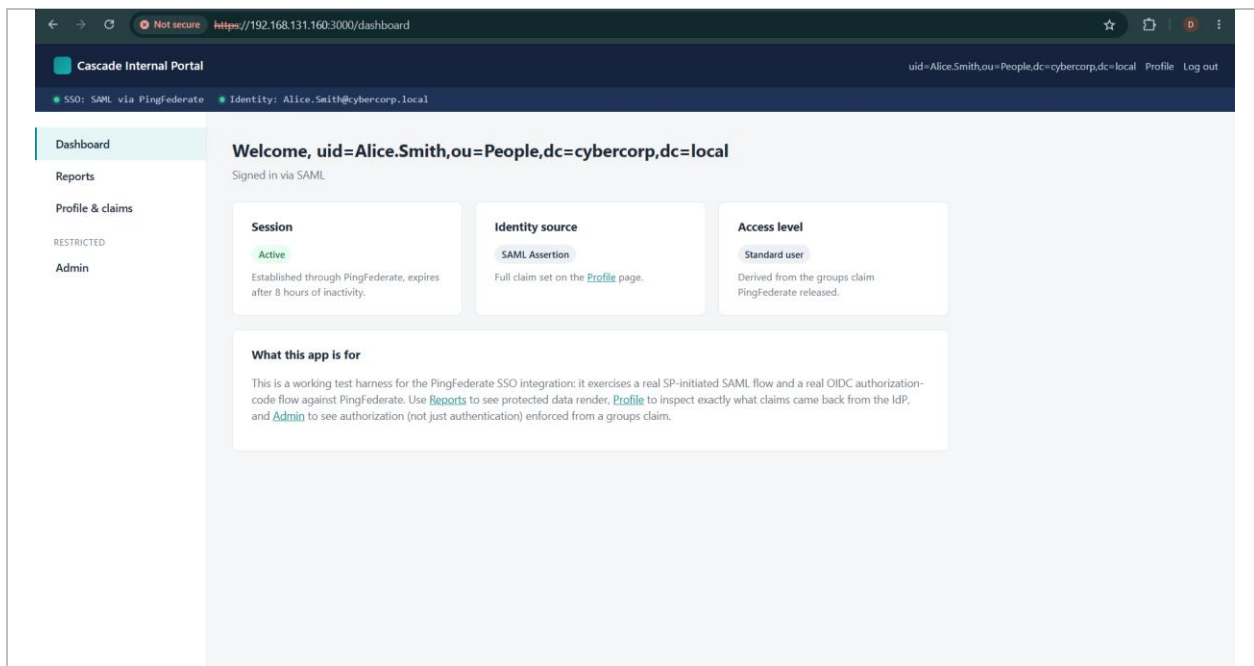


Figure 59. Successful SAML login - the Cascade dashboard showing Alice Smith's identity, signed in via SAML.

What we did and why

With both issues resolved, the full SP-initiated SAML flow was exercised end to end. Starting at the Cascade Internal Portal's own sign-in page (<https://192.168.131.160:3000/login>), which offers a choice between SAML 2.0 (SP-initiated) and OpenID Connect, selecting SAML 2.0 redirected the browser to PingFederate's SSO.saml2 endpoint carrying a SAMLRequest parameter - landing on the newly restyled login form from 4.4. After signing in as Alice Smith, PingFederate authenticated her against PingDirectory through the Lab 2 authentication policy, built and signed a SAML assertion using the Attribute Contract from this lab, and posted it back to Cascade's ACS endpoint.

The final screenshot confirms the whole chain worked: the Cascade dashboard displays "Welcome, uid=Alice.Smith,ou=People,dc=cybercorp,dc=local," labeled "Signed in via SAML," with the top bar showing "SSO: SAML via PingFederate" and "Identity: Alice.Smith@cybercorp.local." The Session card shows an active session established through PingFederate, the Identity source card confirms the identity came from a SAML Assertion, and the Access level card shows a Standard user role. This is the tangible proof that every layer built across Labs 1 through 3 - the PingDirectory identity store, the PingFederate authentication and attribute infrastructure, and this lab's SAML trust relationship - works together as a single, functioning SSO integration.

5. Summary

After completing this lab, the full architecture built across Labs 1 through 3 can be read as a single layered IAM authentication flow. The Cascade Internal Portal acts as the SAML Service Provider and initiates an SSO request when a user needs to authenticate. That request reaches PingFederate, which acts as the Identity Provider and invokes the CyberCorp-Cascade-Authentication policy built in Lab 2. That policy uses the Cascade-Portal-Login HTML Form Adapter and the CyberCorp-LDAP-Validator to check the user's credentials against PingDirectory. Once authentication succeeds, the user's identity and attributes are represented through the Authentication Policy Contract; PingFederate maps those contract values into the SAML assertion built in this lab, places username, uid, givenName, mail, and departmentNumber into it, signs it with the PingFederate signing certificate, and sends the response to Cascade's ACS endpoint. Cascade validates the response and establishes the user's authenticated session.

The three labs form a complete identity chain, each with a distinct responsibility. Lab 1 established PingDirectory as the identity store, where CyberCorp users and their attributes are maintained. Lab 2 established PingFederate's authentication infrastructure - LDAP credential validation, the HTML Form Adapter, the attribute source, the Authentication Policy Contract, and the Authentication Policy itself. Lab 3 established the SAML trust relationship between PingFederate and the Cascade Internal Portal, and defined how authenticated identity is transformed into a SAML assertion the application can consume. PingDirectory provides the identity data, PingFederate performs centralized authentication and identity processing, and Cascade consumes the resulting SAML assertion to provide Single Sign-On - without ever needing to know how PingDirectory or PingFederate's internal authentication actually work.

Along the way, two configuration issues were diagnosed and resolved: a mismatched SP entity ID between the application's SAML_SP_ISSUER and PingFederate's Partner's Entity ID, and PingFederate's global IdP Authentication Policies switch being left off despite a fully configured policy underneath it. Both are now reusable troubleshooting patterns: entity ID mismatches produce a ConfigurationException naming the unrecognized entity ID, while a disabled or unmapped policy engine produces a NoMappedAdapters exception even when the policy itself looks correct in the UI.